Linux Internals in 22 Days

Nov 11, 2026 · @Elad Shechter

Twenty-two sittings, from your first system call to your own drivers. Days 1–4 look at the kernel from outside, on your own Linux machine. Days 5–18 build and boot your own kernel, then write the code each part of it is made of: modules, system calls, the scheduler's data, page tables, allocators, a character device, a PCI driver, a block device, a file system, a network driver and a USB driver, with the debugging tools kernel developers use every day. Days 19–22 add timers and workqueues, containers, a device-tree driver on an ARM board, and eBPF. Every day ends with an exercise you do yourself, and the last chapters cover what else a kernel job asks of you, with a reading list.

How this works

Each day is one sitting of about two hours:

- **Read (30–45 min):** a few exact sections. OSTEP chapters for Days 1–4, then mostly the kernel's own documentation.
- **Build:** the code is in your lab repo, tested on Linux 7.3. Read it with the excerpt the day explains, then `make`.
- **Run:** one block of commands, with what you should see in the comments. Predict before you press Enter.
- **Exercise:** a change you make yourself, 15 to 40 minutes, with the result you should see. A tested answer sits in that day's `solution/` folder; most are a few lines away from the day's code, so `diff` shows them.
- **Check:** two questions. Answers are in the Answer key.

Work on Ubuntu 24.04 installed on your PC, not in WSL:

- Your lab VM gets hardware virtualization (KVM) directly, so it boots in a few seconds and runs at full speed. The emulated test machine needed about 17 seconds per boot.
- USB devices plug straight in. That matters for the ESP32-S3 in After Day 22; in WSL every device must first be passed through with usbipd.
- From Day 5 on, all kernel code runs in a VM, so your own machine's kernel is never at risk. Days 1–4 only read it, through `/proc` and `strace`, and Day 22's eBPF programs are checked by the kernel before they run.

The VM workflow makes Days 5–21 fast:

- `~/os/lab` is shared into the VM as `/lab`: edit and `make` on your machine, then `insmod /lab/...` in the VM. No reboot between builds.
- The kernel's bug detectors (KASAN, lockdep, the sleep-in-atomic check) are on from Day 5, so a mistake in your code gets reported on the spot instead of corrupting memory quietly.
- A crash only takes down the VM. Run `run.sh` again.

Kernel code is C, not C++, with no standard library. Programs stay C++14. **Windows:** lines map each idea to the Windows kernel (WDM/KMDF).

All code and commands were run on Linux 7.3 with GCC 13.3 and QEMU 8.2.2 on Ubuntu 24.04. In-kernel APIs change between releases, so stay on 7.3 for this course.

Setup (once, ~15 min)

Ubuntu 24.04 LTS, installed on your PC (dual boot is fine).

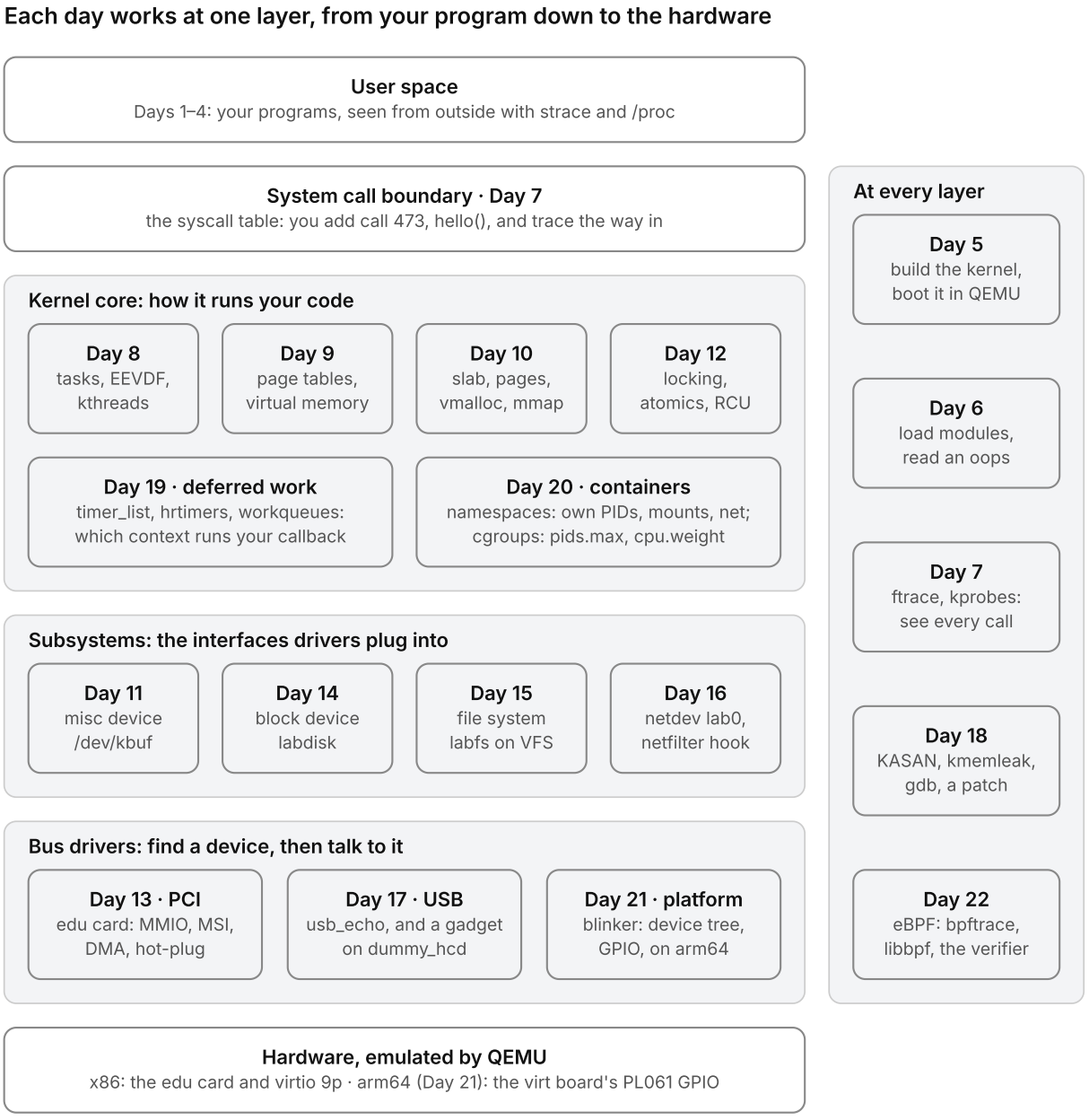
```
sudo apt update
sudo apt install -y build-essential flex bison bc libelf-dev libssl-dev cpio git \
    qemu-system-x86 busybox-static gdb kmod python3-ply python3-git strace
mkdir -p ~/os
git clone -b ccr-77cd2602-bgjf5z https://github.com/eladpipi-sketch/Linux-kernel.git ~/os/lab
qemu-system-x86_64 -device help | grep edu   # name "edu", bus PCI: Day 13's card
ls -l /dev/kvm               # must exist; if not, enable VT-x or AMD-V in the BIOS
sudo usermod -aG kvm $USER   # then log out and back in, so run.sh can use KVM
```

Then, in a second terminal, start your kernel and leave it compiling while you do Days 1–4:

```
~/os/lab/setup/build-kernel.sh && ~/os/lab/setup/make-rootfs.sh   # 10-30 min, depending on
your CPU
```

The lab holds `setup/` (build, root file system and run scripts) and one folder per day, `day01/` to `day22/` (Day 5 uses `setup/`), each with its sources, a `Makefile`, and the exercise's answer in `solution/`.

The map



the course by layer · 22 days

Read it top down. A call from your program crosses the system call boundary, runs on the kernel core, passes through a subsystem, and ends in a bus driver that talks to a device. Days 1–4 watch the top layer from outside; the right column is the toolkit you use at every layer.

Days 1–4: the kernel from outside

Four days on your own machine, no kernel code yet. Everything a program does goes through the kernel; these days make that visible, and the last command of each Run peeks inside. Each day's programs are in `~/os/lab/dayNN`; `make` builds them.

Day 1 — Processes and system calls

Goal: see that a program can do nothing without the kernel: how processes are made, what a system call costs, and what a sleeping process looks like from inside.

Read: OSTEP chapters 4 Processes, 5 Process API and 6 Limited Direct Execution.

Build: `cd ~/os/lab/day01 && make` builds `fork_exec` (fork, exec and wait, the steps behind every program launch) and `syscall_cost` (a million `getppid()` calls against a million plain function calls).

Run:

```
strace -c ls > /dev/null               # every system call ls made, counted
strace ls 2>&1 >/dev/null | tail -4    # the last few, one per line
grep -E 'Name|State|PPid|Threads' /proc/$$/status
ls -l /proc/$$/fd                      # the shell's open files
./fork_exec
strace -f -e trace=process ./fork_exec
./syscall_cost
sleep 60 & sudo cat /proc/$!/stack     # where a sleeping process waits, inside the kernel
cat /proc/cmdline; ls /sys/class/net   # the kernel's own state, as files
```

Notice:

- `ls` made about 70 system calls to list a folder: `getdents64` reads the directory, one `write(1, …)` prints every name, `exit_group` ends the process. Printing touches hardware, so even that is a request to the kernel.
- `/proc/$$/status` is the kernel's record of your shell, read as a text file. `State: S (sleeping)`: the shell waits for `grep` to finish.
- `strace -f` shows fork as `clone(…SIGCHLD…)`, then the child's `execve("/usr/bin/ls", …)` replacing its program, and the parent's `wait4` returning when it exits.
- A system call costs about 100 ns (130 ns here) and a function call about 1 ns: the CPU switches to kernel mode and back, and security mitigations add to it.
- The sleeping process's kernel stack is `hrtimer_nanosleep` ← `common_nsleep` ← `__x64_sys_clock_nanosleep` ← `do_syscall_64` ← `entry_SYSCALL_64_after_hwframe`. A sleeping process is a kernel stack parked inside a system call. From Day 5 on, you'll write code that runs there.

Exercise — your own shell (~30 min): write `tinysh.cpp`: print a prompt, read a line with `std::getline`, split it into words, and `fork()`; the child calls `execvp()`, the parent calls `waitpid()` and prints the exit status. Add `cd` and `exit` as built-ins. Build it with `g++ -std=c++14 -Wall -o tinysh tinysh.cpp`, run it under `strace -f -e trace=process,chdir ./tinysh`, and type `ls`, then `cd /tmp`. `ls` shows `clone`, `execve`, `exit_group` and `wait4`; `cd /tmp` shows a single `chdir("/tmp")` in the shell itself. Why must `cd` be built in? Answer: `day01/solution/tinysh.cpp`.

Windows: system calls go through ntdll.dll into the kernel the same way; `CreateProcess` does fork and exec in one call; Process Monitor is the closest thing to `strace`, and Process Explorer to `/proc`.

Check: (a) What exactly happens between your `getppid()` call and the kernel's answer? (b) Why does Unix split process creation into `fork` and `exec`?

Day 2 — Memory from user space

Goal: see that every pointer is a virtual address, and that the kernel hands out physical memory only when you touch it.

Read: OSTEP chapters 13 Address Spaces, 14 Memory API, 18 Paging: Introduction, and the Linux half of 23 Complete Virtual Memory Systems.

Build: `cd ~/os/lab/day02 && make`: `address_space` (prints where code, globals, heap and stack live), `alloc` (one small and one big `malloc`), `rss` (`malloc` 200 MB, then touch it), `count_lines` (counts a file's lines without `read()`).

Run:

```
./address_space & sleep 1; cat /proc/$!/maps | head -12
strace -e trace=brk,mmap,munmap ./alloc 2>&1 | tail -4
./rss
seq 1000000 > numbers.txt
strace -y -e trace=read,mmap ./count_lines numbers.txt 2>&1 | tail -3
seq 30000000 > big.txt                                  # about 250 MB
sudo sh -c 'sync; echo 3 > /proc/sys/vm/drop_caches'   # empty the page cache
time cat big.txt > /dev/null; time cat big.txt > /dev/null
free -h; rm big.txt numbers.txt
```

Notice:

- Each printed address falls in one line of `/proc/<pid>/maps`: code in your binary's `r-xp` line, the global in its `rw-p` line, the heap in `[heap]`, the local in `[stack]`. Run it twice: ASLR moves everything.
- `brk` grows the heap for the 100-byte block; the 1 MB block gets its own `mmap(NULL, 1052672, …)` (1 MB plus one 4 KB page of bookkeeping), and `free` hands it straight back with `munmap`.
- `VmRSS` barely moves after `malloc` (3376 kB to 3512 kB here) and jumps by 200 MB after `memset` (208312 kB): `malloc` only reserved address space; RAM arrived page by page as `memset` touched it.
- `count_lines` maps the file and never `read()`s it, yet counts a million lines: page faults bring the file in.
- The second `cat` was several times faster, and `buff/cache` grew by about the file's size: the page cache.

Exercise — count page faults (~30 min): write `faults.cpp`: `mmap` 64 MB of anonymous memory, write one byte every 4096 bytes, and print how many minor page faults that took (`getrusage(RUSAGE_SELF, ...)`, field `ru_minflt`, before and after). Predict, then run. Then write every 8192 bytes. Then start the region on a 2 MB boundary, call `madvise(start, size, MADV_HUGEPAGE)` first, and write every 4096 bytes again:

```
every 4 KB:               16384 faults
every 8 KB:                8192 faults
every 4 KB, huge pages:      32 faults
```

One fault per 4 KB page (64 MB / 4 KB = 16,384), and one per 2 MB once you ask for transparent huge pages. Ubuntu's default THP mode is `madvise` (see `/sys/kernel/mm/transparent_hugepage/enabled`). Answer: `day02/solution/faults.cpp`.

Windows: `VmRSS` is the working set; the heap and fresh pages are `HeapAlloc` and `VirtualAlloc`; mapping a file is `CreateFileMapping` + `MapViewOfFile`; Sysinternals VMMap shows what `/proc/<pid>/maps` does.

Check: (a) Why does `malloc` give big blocks their own `mmap`, but carve small ones from the heap? (b) `count_lines` never called `read()`. How did the file's bytes get into memory, and why was the second `cat` faster?

Day 3 — Threads, locks and signals

Goal: see what threads, mutexes and signals look like from the kernel's side, and catch a deadlock in the act.

Read: OSTEP chapters 26 Concurrency: An Introduction, 28 Locks (including its section on Linux futexes) and 32 Common Concurrency Problems; `man 7 signal`, "Signal dispositions" and the "Standard signals" table.

Build: `cd ~/os/lab/day03 && make`: `threads` (three sleeping threads), `race` (two threads add a million each to one counter: `./race` without a lock, `./race mutex` with one), `signals` (catches SIGINT and SIGTERM), `deadlock` (two threads take two locks in opposite orders).

Run:

```
./threads & sleep 1; ls /proc/$!/task; kill $!
timeout 2 strace -f -e trace=clone3 ./threads 2>&1 | grep -m1 clone3
./race; ./race; ./race mutex
strace -f -c -e trace=futex ./race mutex
./signals & sleep 1; grep SigCgt /proc/$!/status; kill -TERM $!
./signals & sleep 1; kill -KILL $!
./deadlock & sleep 1; grep . /proc/$!/task/*/wchan
sudo cat /proc/$!/task/*/stack | grep -m3 futex; kill -9 $!
```

Notice:

- Four entries in `/proc/<pid>/task`: a thread is a task with its own ID, made by `clone3(… CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD…)`. Day 1's fork had no `CLONE_VM`: a thread shares the address space, a child process gets its own.
- Without the lock the count came out near 1.2 million instead of 2 million. With it the count is right, and strace counts only a few thousand `futex` calls for two million locks: the kernel gets involved only when a thread has to wait.

- `SigCgt: 0000000000004002` is a bitmask of caught signals: bit 1 is SIGINT (2), bit 14 is SIGTERM (15). `kill -TERM` is caught (`got signal 15, exiting cleanly`); `kill -KILL` just says `Killed`, and no handler runs.
- Every thread of the deadlocked program sleeps in `futex_do_wait` (`futex_wait_queue` on older kernels), with the kernel stack `futex_do_wait` ← `__futex_wait` ← `futex_wait`: blocked inside the kernel, forever.

Exercise — a spinlock (~30 min): add a third mode to `race.cpp`: `./race spin` protects the counter with a spinlock you build from `std::atomic_flag`: lock with `while (flag.test_and_set(std::memory_order_acquire)) ;`, unlock with `flag.clear(std::memory_order_release)`. Compare `time ./race mutex` with `time ./race spin`, and their `strace -f -c -e trace=futex` counts. Here the spinlock made no futex calls and used no system time, and on 4 CPUs it finished first (0.11 s against 0.25 s): for a critical section of a few nanoseconds, spinning beats sleeping and waking through the kernel. Day 12 meets the same trade-off inside the kernel. Answer: `day03/solution/race.cpp`.

Windows: Linux makes processes and threads with the same call; `CRITICAL_SECTION` and `SRWLOCK` follow the futex plan (an atomic operation in user space, a kernel wait only on contention, `WaitOnAddress`); Ctrl+C maps to `SetConsoleCtrlHandler`; Visual Studio's Parallel Stacks shows the same deadlock picture.

Check: (a) Why is the unlocked count wrong, and why does an uncontended mutex never enter the kernel? (b) Why does the signal handler only set a flag, and why can't SIGKILL be caught?

Day 4 — Files and I/O

Goal: see what a file descriptor, an inode, `fsync` and an event loop really are.

Read: OSTEP chapters 39 Files and Directories, 40 File System Implementation (the inode sections), 42 Crash Consistency: FSCK and Journaling (first half) and 33 Event-based Concurrency; skim `man 7 epoll`.

Build: `cd ~/os/lab/day04 && make`: `fd` (opens a file and keeps it open), `writes` (1,000 log lines, with or without `fsync`), `epoll_stdin` (one thread waits for input with epoll).

Run:

```
./fd & sleep 1; ls -l /proc/$!/fd; kill $!
strace -f -e trace=openat,dup2 bash -c 'echo hi > /tmp/out.txt'
mkdir -p ~/os/links && cd ~/os/links
echo hello > a.txt; ln a.txt hard.txt; ln -s a.txt soft.txt; ls -li
rm a.txt; cat hard.txt; cat soft.txt              # hello; No such file or directory
sleep 300 < hard.txt & sleep 1; rm hard.txt
ls -l /proc/$!/fd/0; cat /proc/$!/fd/0; kill $!   # (deleted), yet still readable
cd ~/os/lab/day04; ./writes; ./writes sync; rm log.txt
sleep 100 | ./epoll_stdin & sleep 1
cat /proc/$(pidof epoll_stdin)/fdinfo/3           # the interest list, inside the kernel
sudo cat /proc/$(pidof epoll_stdin)/stack; kill %1
```

Notice:

- `fd` got descriptor 3: 0, 1 and 2 are stdin, stdout and stderr, and the kernel always hands out the lowest free number. It points at `/usr/lib/os-release`, because `/etc/os-release` is a symlink to it.
- Redirection is `openat("/tmp/out.txt") = 3`, then `dup2(3, 1)`: the program writes to fd 1 as usual and never knows.
- `a.txt` and `hard.txt` share one inode number, with a link count of 2; `soft.txt` is its own inode that stores a name. A deleted file lives until its last name and its last open descriptor are gone.
- 1,000 plain writes take 0 ms; with `fsync` after each one they take orders of magnitude longer (159 ms on the test machine's virtual disk).
- `fdinfo` shows the epoll interest list as the kernel keeps it: `tfd: 0 events: 19`, watching fd 0 for `EPOLLIN|EPOLLERR|EPOLLHUP` (0x19). The waiting thread sleeps in `do_epoll_wait`.

Exercise — redirection (~20 min): give Day 1's `tinysh` the `>` operator: when a line contains `> file`, the child opens the file (`O_WRONLY | O_CREAT | O_TRUNC`, mode 0644), moves it onto fd 1 with `dup2(fd, 1)` and closes the original, all before `execvp()`. Run it under `strace -f -e trace=openat,dup2,execve ./tinysh` and type `echo hi > out.txt`: the child's `openat` returns 3, then `dup2(3, 1)`, then `execve`, exactly what bash did above. Answer: `day04/solution/tinysh.cpp`.

Windows: an fd is a `HANDLE`, both indexes into a per-process table of kernel objects; NTFS has hard links (`mklink /H`), but usually refuses to delete a file that's open; `fsync` is `FlushFileBuffers`; IOCP is the completion-based counterpart of epoll.

Check: (a) Your program writes a log line and the power fails one second later. Is the line on disk, and what would guarantee it? (b) A server's disk is full, but `du` can't find what's using the space. What's going on, and how do you find it?

Days 5–7: your kernel, your code, the system call boundary

Day 5 — Build it, boot it, watch it start

Goal: a kernel you compiled, booting in a VM whose every piece you know, with the bug detectors on.

Read: Linux Kernel Labs Introduction, the "Overview of the Linux kernel" part; Ramfs, rootfs and initramfs, "What is initramfs?".

Build: setup ran the first two scripts, and the third boots the result. Read all three; each is a dozen lines:

```
~/os/lab/setup/build-kernel.sh   # clone v7.3, defconfig + kvm_guest.config + lab.config,
compile (15-30 min)
~/os/lab/setup/make-rootfs.sh    # the VM's whole user space: static BusyBox + a 12-line init
~/os/lab/setup/run.sh            # boot it; type exit to power off
```

`lab.config` adds debug info, ftrace, KASAN, lockdep, kmemleak and a software USB controller. In `run.sh`, `-kernel`/`-initrd` boot your kernel directly with no bootloader, `-append` is the kernel command line, `-device edu,addr=4` is Day 13's PCI card, `-virtfs` shares `~/os/lab` as `/lab`, and `-s` opens a gdb port for Day 18.

Run:

```
uname -r; cat /proc/cmdline        # 7.3.0; your arguments, as the kernel got them
zcat /proc/config.gz | grep -E "^CONFIG_(KASAN|PROVE_LOCKING)="
mount | grep -E "9p|tracefs"        # lab on /lab type 9p
ps | head -4                        # PID 1 is the init script; [names] are kernel threads
ls /sys/bus/pci/devices             # 0000:00:04.0 is the edu card
exit
~/os/lab/setup/run.sh initcall_debug    # boot again, logging every init function
dmesg | grep -c "initcall .* returned"
dmesg | grep "initcall .* returned" | awk '{print $(NF-1), $4}' | sort -n | tail -5
dmesg | grep "Run /init"
```

Then, on your machine (not in the VM), measure what the kernel is made of:

```
cd ~/os/linux
git ls-files '*.[ch]' | xargs cat | wc -l                      # 37.7 million lines
git ls-files 'drivers/*.[ch]' | xargs cat | wc -l              # 26.4 million: drivers
git ls-files 'kernel/*.[ch]' 'mm/*.[ch]' | xargs cat | wc -l   # 0.8 million: scheduler,
memory, ...
```

Notice:

- 728 init functions ran before the kernel started `/init`. On the emulated test machine that took 13 seconds, and the slowest were `acpi_init` (3.5 s), `param_sysfs_builtin_init` (1.4 s) and `slab_late_init` (0.9 s), in microseconds in the second column. Under KVM yours are far faster, and the slowest few may differ.
- 70% of the kernel is drivers; the scheduler and memory manager are 2%.
- With KASAN and debug info, `bzImage` is 33 MB and `vmlinux` 533 MB: the second one is what gdb and `faddr2line` read.

Exercise — be init (~15 min): boot without the init script, with BusyBox's shell as PID 1:

```
~/os/lab/setup/run.sh rdinit=/bin/busybox -- sh
echo $$                             # 1
ls /proc /sys /lab                  # all empty: nothing is mounted
mount -t proc proc /proc; ps        # now ps works
exit
```

`rdinit=` names the first program the kernel runs from an initramfs, and everything after `--` becomes its arguments. Mounting `/proc` is one of the jobs `setup/init` normally does; you just did it by hand. The final `exit` prints `Kernel panic - not syncing: Attempted to kill init!`

`exitcode=0x00000000`; `panic=-1` reboots at once, and QEMU's `-no-reboot` closes the VM. That's Check (b), seen live.

Windows: Windows boots winload → ntoskrnl.exe → smss.exe; here QEMU loads your kernel straight into memory, and `/init` is the first user process.

Check: (a) What does `.config` decide, and why not build every driver into the kernel? (b) PID 1 is a shell script. What happens if it exits, and why does it end with `poweroff -f`?

Day 6 — Modules, kernel C, and your first oops

Goal: load your own code into the running kernel, and learn to read a kernel crash.

Read: LKMPG sections 4.1–4.5; Unreliable Guide To Hacking The Linux Kernel: "The Players", "Some Basic Rules", "Recipes for Deadlock"; the Memory Allocation Guide.

Build: `cd ~/os/lab/day06 && make`. Three modules, each under 60 lines:

- `hello.c`: `module_init`/`module_exit`, and a parameter that shows up as a file: `module_param(whom, charp, 0644)`.
- `crash.c`: writes through a NULL `struct sensor *`. Its `Makefile` line `KASAN_SANITIZE_crash.o := n` turns KASAN off for this one file so you see the classic report.
- `list.c`: `kmalloc`, a `struct list_head` embedded in each item, and `container_of` to get from the link back to the item, the kernel's version of a C++ base-class cast.

Run:

```
cd /lab/day06
insmod hello.ko; cat /sys/module/hello/parameters/whom      # hello, world / world
echo kernel > /sys/module/hello/parameters/whom; rmmod hello   # goodbye, kernel
insmod hello.ko whom=Linux
grep hello /proc/modules           # hello 12288 0 - Live 0xffffffffc0000000 (O)
grep '\[hello\]' /proc/kallsyms     # its symbols, now part of the kernel's
rmmod hello; cat /proc/sys/kernel/tainted                     # 4096
insmod list.ko; rmmod list
insmod crash.ko                    # read the whole report
echo still alive; cat /proc/sys/kernel/tainted               # 4224
# then, on your machine:
cd ~/os/linux && ./scripts/faddr2line ~/os/lab/day06/crash.ko crash_init+0x17
```

Notice:

- The report starts `BUG: kernel NULL pointer dereference, address: 0000000000000004`, then `#PF: supervisor write access in kernel mode` and `RIP: 0010:crash_init+0x17`. `faddr2line` answers `crash.c:14`. `insmod` is `Killed`; the shell lives on.
- `list.c` prints its items at `ffff8880...` (the kernel's map of all RAM) and its code at `ffffffffc0...` (where modules load). Every kernel address starts with `ffff`: the top half of every address space is the kernel.
- `tainted` went 4096 (O: out-of-tree module) → 4224 (+128, D: the kernel has oopsed).

Exercise — a sorted list (~30 min): change `list.c` so it takes its values from a parameter, `module_param_array(values, int, &nr_values, 0444)`, and inserts each one in sorted order: walk with `list_for_each_entry()` and use `list_add_tail(&new->node, &pos->node)`, which puts `new` just before `pos`. Then delete every item equal to a `drop=` parameter with `list_for_each_entry_safe()`, printing the list before and after (`pr_info` for the start of the line, `pr_cont` for each value). `insmod list.ko values=50,10,40,20 drop=40` should print `list: sorted: 10 20 40 50` and `list: after dropping: 10 20 50`. Why does deleting need the `_safe` walk? Answer: `day06/solution/list.c`.

Windows: a `.ko` is a `.sys`, `module_init`/`module_exit` are `DriverEntry`/`DriverUnload`, `pr_info` is `DbgPrint`, and an oops is a bugcheck the kernel survived.

Check: (a) Why is the faulting address 4 and not 0, and why didn't the VM die? (b) When must you use `GFP_ATOMIC` instead of `GFP_KERNEL`?

Day 7 — System calls from the inside

Goal: follow a system call into the kernel, watch them without writing code, and add one.

Read: Adding a New System Call: "System Call Alternatives", "Generic System Call Implementation", "x86 System Call Implementation"; Kprobe-based Event Tracing: "Overview" and "Synopsis of kprobe_events".

Build: `cd ~/os/lab/day07 && make` builds two tiny C++ programs: `getppid_test` (one `getppid()`, Day 1's call) and `syscall_test` (calls number 473).

Run: three views of the same boundary.

```
cd /sys/kernel/tracing
# 1. the kernel functions behind one getppid()
echo __do_sys_getppid > set_graph_function
echo 0 > options/funcgraph-irqs          # leave out interrupts that land mid-call
echo function_graph > current_tracer
/lab/day07/getppid_test
grep -A7 "__do_sys_getppid() {" trace
echo nop > current_tracer; echo > set_graph_function; echo 1 > options/funcgraph-irqs
# 2. every getppid on the machine, by number (110 on x86-64)
echo 'id == 110' > events/raw_syscalls/sys_enter/filter
echo 1 > events/raw_syscalls/sys_enter/enable
/lab/day07/getppid_test
echo 0 > events/raw_syscalls/sys_enter/enable; echo 0 > events/raw_syscalls/sys_enter/filter
grep sys_enter trace
# 3. a probe on any kernel function: log every file anyone opens
echo 'p:lab_open do_sys_openat2 path=+0(%si):ustring' > kprobe_events
echo 1 > events/kprobes/lab_open/enable
cat /lab/day07/Makefile > /dev/null; ls / > /dev/null
echo 0 > events/kprobes/lab_open/enable
grep lab_open trace
echo '-:lab_open' >> kprobe_events
```

Now add system call 473. On your machine:

```
cd ~/os/linux
git apply ~/os/lab/day07/hello-syscall.diff   # one table line + SYSCALL_DEFINE1(hello, int,
x)
time make -j$(nproc)                          # most of the kernel recompiles: ~13 min on 4
cores
for d in ~/os/lab/day*/; do make -C $d; done  # see Notice
```

The whole change: the line `473 common hello sys_hello` in `arch/x86/entry/syscalls/syscall_64.tbl`, and in `kernel/sys.c`:

```
SYSCALL_DEFINE1(hello, int, x)
{
    pr_info("hello: called by %s (pid %d) with %d\n", current→comm, current→pid, x);
    return x * 2;
}
```

Reboot the VM and run `/lab/day07/syscall_test` (`hello(21) returned 42`), then `dmesg | tail -1`.

Notice:

- The graph shows `__do_sys_getppid()` calling `__task_pid_nr_ns()` between `__rcu_read_lock()`/`__rcu_read_unlock()`: your parent is read under RCU (Day 12) and translated into your PID namespace.
- `sys_enter: NR 110 (...)` shows the raw syscall number and the six argument registers.
- The kprobe caught `path="/lab/day07/Makefile"` from `cat` and `path="/"` from `ls`, on a function you never touched.
- After the change, `uname -r` says `7.3.0-dirty`: the build stamps uncommitted git changes into the version, so modules built before are refused with `version magic '7.3.0 ...' should be '7.3.0-dirty ...'`. That's why you rebuilt them.
- `current` is the calling task's `task_struct`: a system call runs in the caller's context.

Exercise — what did open return? (~20 min): add a return probe next to the entry probe, `r:lab_open_ret do_sys_openat2 ret=$retval:s32`, enable both (`echo 1 > events/kprobes/enable`), then run `cat /lab/day07/Makefile > /dev/null` and `cat /no/such/file`. In the trace, each `path="..."` line is followed by its result: `ret=3` for the file that opened, `ret=-2` (`ENOENT`) for the missing one. A return probe reads the raw return register, and `do_sys_openat2()` returns `int` in 7.3, so read it as `s32`: as `s64`, the -2 shows up as 4294967294. Answer: `sh /lab/day07/solution/open_probes.sh`.

Windows: the table you edited is the counterpart of the SSDT; kprobe events are roughly what ETW kernel providers give you, but on any function.

Check: (a) `__do_sys_getppid` can be traced, but `do_syscall_64` can't even be selected. Why? (b) Why do drivers almost never add a system call, and what do they use instead?

Days 8–10: processes and memory, from the kernel's side

Day 8 — Tasks and the scheduler

Goal: see processes as the kernel stores them, start a kernel thread, and watch the scheduler split a CPU.

Read: Linux Kernel Labs Processes: "Processes and threads", "Context switching", "Blocking and waking up tasks"; EEVDF Scheduler, the fair scheduler since 6.6.

Build: `cd ~/os/lab/day08 && make`.

- `tasks.c` walks every `struct task_struct` with `for_each_process(p)` under `rcu_read_lock()`, printing `p→pid`, `p→comm`, `p→__state`, `p→prio`, `p→se.vruntime` and the switch counts.
- `worker.c` starts a kernel thread with `kthread_run()`; it loops `while (!kthread_should_stop())` and sleeps with `msleep_interruptible(1000)`. `kthread_stop()` in the exit function wakes it and waits for it to return.

Run:

```
cd /lab/day08
insmod tasks.ko; rmmod tasks
insmod worker.ko; sleep 3; ps | grep lab_worker
grep -E "State|ctxt" /proc/$(pidof lab_worker)/status; rmmod worker
# two CPU hogs on CPU 1, and B gets nice 19 at once: keep the next three on one line
taskset -c 1 yes > /dev/null & A=$!; taskset -c 1 yes > /dev/null & B=$!; renice -n 19 -p $B
sleep 5; grep -E "^(se.vruntime|se.sum_exec_runtime|nr_switches|prio)" /proc/$A/sched
/proc/$B/sched
cd /sys/kernel/tracing
echo 1 > events/sched/sched_switch/enable; sleep 1; echo 0 > events/sched/sched_switch/enable
grep "prev_comm=yes" trace | tail -3
kill $A $B; cd /lab
```

Notice:

- `tasks` lists everything, kernel threads included. `__state` 0x1 is S (sleeping), 0x402 is I (an idle kernel worker), 0x0 is running: the caller itself, `insmod`. `prio` is 120 for normal tasks, 139 for nice 19, 100 for nice −20 kernel workers, and 0 for the per-CPU `migration/N` stop tasks.
- `[lab_worker]` sleeps in state S, and every tick is a voluntary switch.
- After 5 seconds the nice-0 `yes` had 5,054 ms of CPU and the nice-19 one 89 ms, yet their `se.vruntime` values were almost equal (11,101 vs 11,173 ms). The scheduler equalizes virtual time; nice only changes how fast it runs.
- `sched_switch` shows the slices: nice 0 ran about 119 ms, nice 19 about 1.7 ms, then they swap.

Exercise — a task's family (~30 min): give `tasks.c` a `pid=` parameter. When it's set, find that task with `pid_task(find_vpid(pid), PIDTYPE_PID)` under `rcu_read_lock()`, then follow `p→real_parent` up to PID 0, printing each task's pid, `comm`, `task_nice(p)` and `se.sum_exec_runtime` in milliseconds. `insmod tasks.ko pid=$$` printed:

```
tasks:    73 sh              nice   0  cpu 410 ms
tasks:     1 init            nice   0  cpu 10704 ms
tasks:     0 swapper/0       nice   0  cpu 0 ms
```

PID 0 is the idle task, everyone's ancestor; its parent is itself, so stop there. Answer: `day08/solution/tasks.c`.

Windows: `task_struct` merges EPROCESS and ETHREAD; a kernel thread is `PsCreateSystemThread`; nice is the base priority, but Linux shares the CPU by weight, not by strict priority levels.

Check: (a) Two nice-0 processes both get 50% of a CPU. Why does nice 19 against nice 0 give under 2% here, and does a nice-19 process run slower on an idle CPU? (b) What is the difference between state S and state I?

Day 9 — Page tables, by hand

Goal: translate a virtual address to a physical one twice, once through `/proc` and once by walking the page tables yourself, and catch the page fault in between.

Read: Page Tables; Examining Process Page Tables, the `/proc/pid/pagemap` part; skim the table in x86-64 memory map.

Build: `cd ~/os/lab/day09 && make`.

- `pagemap_test.cpp` maps one fresh page, asks `/proc/self/pagemap` about it before and after the first write, then sleeps.
- `pagewalk.c` takes a `pid` and `addr`, then walks `pgd_offset()` → `p4d_offset()` → `pud_offset()` → `pmd_offset()` → `pte_offset_kernel()` under `mmap_read_lock()`, printing each entry.

The safe last step, `pte_offset_map()`, isn't exported to modules, so it would fail to link (`modpost: symbol '__pte_offset_map' undefined`). On x86-64, page-table pages live in the kernel's direct map, so `pte_offset_kernel()` finds the same entry.

Run:

```
cd /lab/day09
./pagemap_test &                 # pid 74, page at 0x7f08a5211000 ...
insmod pagewalk.ko pid=74 addr=0x7f08a5211000   # use the numbers it printed
rmmod pagewalk; kill %1
cd /sys/kernel/tracing
echo 1 > events/exceptions/page_fault_user/enable
/lab/day09/pagemap_test & sleep 2; kill $!
echo 0 > events/exceptions/page_fault_user/enable
grep pagemap_test trace | tail -4
```

Notice:

- `before touch: present=0`, `after touch: present=1 pfn=0x6eeaa`. The write caused a page fault that allocated the page.

- The walk printed `pte entry 0x800000006eeaa067` and `→ pfn 0x6eeaa → physical 0x6eeaa000`, the same frame as pagemap. In `...067`, 0x1 is present, 0x2 writable, 0x4 user, 0x20 accessed and 0x40 dirty. Bit 63 is NX: no code runs from this page.
- The fault trace shows `address=0x7f822ed53000 ... error_code=0x6` for your write (2 = write, 4 = user mode, page not present) among faults with `error_code=0x14` (0x10 = instruction fetch, 4 = user mode): the program's code being paged in on demand.

Exercise — page sizes (~30 min): make `pagewalk` report the page size and physical address wherever the walk stops: at a PUD leaf, a 1 GB page (`pud_pfn()`, offset `addr & ~PUD_MASK`); at a PMD leaf, 2 MB (`pmd_pfn()`, `~PMD_MASK`); at the PTE, 4 KB. Then walk two kernel addresses through your shell's page tables, since the kernel's half is mapped in every process:

```
A=0x$(grep " T _printk$" /proc/kallsyms | cut -d" " -f1); insmod pagewalk.ko pid=$$ addr=$A;
rmmod pagewalk
A=0x$(grep " T schedule$" /proc/kallsyms | cut -d" " -f1); insmod pagewalk.ko pid=$$ addr=$A;
rmmod pagewalk
grep DirectMap /proc/meminfo
```

Here `_printk` ended on a 2 MB page and `schedule` on a 4 KB one. The kernel maps its code and most of RAM with big pages (`DirectMap2M` about 2 GB, `DirectMap4k` 20 MB; with KVM you may also see `DirectMap1G`), but `schedule` sits in the last 2 MB of kernel code, which the kernel split into 4 KB pages when it freed the gap between the end of code and the start of read-only data. Answer: `day09/solution/pagewalk.c`.

Windows: the same 4-level x86-64 tables, walked by `MmGetPhysicalAddress` for kernel addresses; RAMMap shows the physical side.

Check: (a) Why does the walk stop at the PMD for some addresses, and what does `pmd_leaf()` mean? (b) Two processes print the same virtual address. How can the walk give different physical pages?

Day 10 — Kernel memory, and sharing it with user space

Goal: use the kernel's three allocators, see where each one's memory comes from, and map a kernel page straight into a process.

Read: Linux Kernel Labs Memory Management: "Physical Memory Management" (buddy and slab allocators); Linux Kernel Labs Memory mapping: "Background information".

Build: `cd ~/os/lab/day10 && make`.

- `slabs.c`: its own slab cache (`kmem_cache_create("lab_obj", ...)`, 1,000 objects), eight pages from the buddy allocator (`alloc_pages(GFP_KERNEL, 3)`), and a check of how many pages behind `kmalloc(1 MB)` and `vmalloc(1 MB + 4 KB)` sit next to each other in RAM.
- `labmap.c`: a misc device whose `.mmap` puts one kernel page into the caller's page tables with `vm_insert_page()`; `labmap_test.cpp` reads it and writes back without `read()` or `write()`.

Run:

```
cd /lab/day10
insmod slabs.ko
grep lab_obj /proc/slabinfo       # lab_obj 1008 1020 272 15 1 : ...
cat /proc/buddyinfo               # free blocks of 1, 2, 4 ... 1024 pages, per zone
rmmod slabs
insmod labmap.ko; ./labmap_test   # the kernel wrote: hello from the kernel
rmmod labmap                      # labmap: the page now says: hello from user space
```

Notice:

- Your 204-byte objects take 272 bytes each in `/proc/slabinfo`: KASAN's red zones and metadata. 15 fit in one 4 KB slab page.
- `kmalloc 1MB: 256 of 256 pages contiguous; vmalloc 1MB+4KB: 256 of 257`. kmalloc memory is always one physical block. In 7.3, vmalloc first grabs big blocks too, but only promises contiguous virtual addresses: the 257th page came from somewhere else.
- `kmalloc` addresses start `ffff8880...` (direct map), `vmalloc` ones `ffffc900...` (the vmalloc area): two regions of the table you skimmed yesterday.
- `labmap` never copied anything: the page sat in both address spaces at once.

Exercise — pages on demand (~40 min): make `labmap` share four pages and map none of them in `mmap()`: set `vma→vm_ops` to a `vm_operations_struct` whose `.fault` handler takes `pages[vmf →pgoff]`, adds a reference with `get_page()`, stores it in `vmf→page`, logs the fault and returns 0 (`VM_FAULT_SIGBUS` past the fourth page). The solution's `fault_test` maps four pages and writes only to pages 0 and 2:

```
labmap: fault on page 0 by fault_test
labmap: fault on page 2 by fault_test
labmap: page 0 says: page 0 was here
labmap: page 1 says: kernel page 1
```

Pages 1 and 3 were never touched, so they were never mapped: Day 2's demand paging, done by your driver. Answer: `day10/solution/`.

Windows: `kmem_cache` is a lookaside list (`ExInitializeLookasideListEx`); `vm_insert_page` is like mapping an MDL into user space with `MmMapLockedPagesSpecifyCache`.

Check: (a) A device does DMA into a 1 MB buffer. Which allocator, and why not `vmalloc`? (b) Why does `vm_insert_page()` beat `read()` for a frame grabber that delivers 30 MB a second?

Days 11–13: writing drivers

Day 11 — A character device, complete

Goal: a device file with every interface a real driver offers: read, write, ioctl, poll, sysfs and debugfs, safe under concurrent use.

Read: LKMPG chapter 6 (character devices) and chapter 12 (blocking processes); ioctl based interfaces: "Command number definitions"; sysfs: "What it is" through "Attributes".

Build: `cd ~/os/lab/day11 && make`. `kbuf.c` is `/dev/kbuf`, a 4 KB pipe in the kernel (146 lines). Its heart is `read()`:

```
    mutex_lock(&kbuf_lock);
    while (data_len == 0) {         /* nothing to read: sleep */
        mutex_unlock(&kbuf_lock);
        if (wait_event_interruptible(kbuf_wait, data_len > 0))
            return -ERESTARTSYS;    /* a signal woke us up */
        mutex_lock(&kbuf_lock);     /* another reader may have won */
    }
    ret = min(count, data_len);
    if (copy_to_user(buf, data, ret))   /* never memcpy to a user pointer */
        ret = -EFAULT;
```

The rest: `write()` appends and calls `wake_up_interruptible()`; `kbuf_ioctl` answers `KBUF_IOC_CLEAR` and `KBUF_IOC_USED` (defined in `kbuf.h` with `_IO`/`_IOR`, shared with `kbuf_test.cpp`); `kbuf_poll` makes it work with epoll; `DEVICE_ATTR_RO(used)` adds a sysfs file; `debugfs_create_u32` exposes a write counter. `file_operations` ties it together, like a hand-written vtable.

Run:

```
cd /lab/day11
insmod kbuf.ko; ls -l /dev/kbuf          # c 10, 258: misc major, dynamic minor
./kbuf_test                              # KBUF_IOC_USED = 0x80046b02 ...
cat /dev/kbuf & sleep 1                  # blocks
cat /proc/$!/wchan; echo                 # kbuf_read
echo hi > /dev/kbuf                      # the background cat prints hi
cat /sys/class/misc/kbuf/used /sys/kernel/debug/kbuf/writes
kill %1
./epoll_test < /dev/kbuf &               # Day 4's event loop, on your device
echo hello > /dev/kbuf                   # got 6 bytes
kill %1; rmmod kbuf
```

Change it: on your machine (not in the VM), swap the mutex for a spinlock and predict what the kernel says:

```
cd ~/os/lab/day11 && cp kbuf.c kbuf.c.good
sed -i 's/DEFINE_MUTEX/DEFINE_SPINLOCK/; s/mutex_lock/spin_lock/g;
s/mutex_unlock/spin_unlock/g' kbuf.c
make        # in the VM: insmod /lab/day11/kbuf.ko; echo hi > /dev/kbuf; rmmod kbuf
mv kbuf.c.good kbuf.c && make
```

Notice:

- `0x80046b02` packs the ioctl's direction (read), size (4), type ('k') and number (2). An unknown command fails with `Inappropriate ioctl for device`, `ENOTTY`.
- The blocked `cat` sleeps in `kbuf_read`, state S, costing no CPU. Without `.poll`, `epoll_ctl` refuses the device with `EPERM`.
- With the spinlock, the write still works, yet the kernel prints `BUG: sleeping function called from invalid context at ./include/linux/uaccess.h:169` and lockdep adds `Invalid wait context`: `copy_from_user` may sleep on a page fault, which is forbidden under a spinlock.

Exercise — don't block (~15 min): make `read()` honor `O_NONBLOCK`: if the buffer is empty and `file->f_flags & O_NONBLOCK`, return `-EAGAIN` instead of sleeping. Test with the solution's `nb_read`, which opens `/dev/kbuf` with `O_NONBLOCK` and reads once: empty, it prints `read: Resource temporarily unavailable`; after `echo hi > /dev/kbuf`, `read 3 bytes: hi`. Every event loop relies on this. Answer: `diff day11/kbuf.c day11/solution/kbuf.c`, two lines.

Windows: `file_operations` is your dispatch table (`IRP_MJ_READ`, `IRP_MJ_WRITE`, `IRP_MJ_DEVICE_CONTROL`); `_IOR` is `CTL_CODE`; the wait queue is a `KEVENT`; a spinlock raises IRQL to `DISPATCH_LEVEL`, where waiting is forbidden.

Check: (a) Why must `read()` check `data_len` again after re-taking the mutex? (b) Why `copy_to_user` and not `memcpy`?

Day 12 — Concurrency: races, atomics, RCU, lockdep

Goal: see a data race on two CPUs, fix it two ways, update shared data with no reader locks, and let lockdep find a deadlock that never happened.

Read: Unreliable Guide To Locking through "Hard IRQ Context"; What is RCU?, sections 1 and 2.

Build: `cd ~/os/lab/day12 && make`.

- `race.c`: two kernel threads pinned to CPU 0 and CPU 1 (`kthread_create` + `kthread_bind`) each add 200,000 to three counters: `WRITE_ONCE(plain, READ_ONCE(plain) + 1)`, `atomic_inc()`, and `++` under a spinlock.
- `rcu_lab.c`: a reader thread loops on `rcu_read_lock(); cfg = rcu_dereference(live_config); ...; rcu_read_unlock()`, while init publishes versions 2 to 4 with `rcu_replace_pointer()`, then `synchronize_rcu()` before freeing the old one.
- `lockorder.c`: takes `lock_a` then `lock_b`, later `lock_b` then `lock_a`, one after the other.

Run:

```
cd /lab/day12
insmod race.ko; rmmod race          # run it twice: the plain count changes
insmod rcu_lab.ko; rmmod rcu_lab
insmod lockorder.ko; rmmod lockorder
```

Notice:

- `race: expected 400000, plain 399698, atomic 400000, spinlock 400000`: about 300 lost updates, a different number each run.

- The reader prints `now sees version 2` before the writer prints `published version 2, freed version 1`: the reader switched over during the grace period, and the old version was freed only after every reader had left it.
- lockdep prints `WARNING: possible circular locking dependency detected` and draws the two-CPU interleaving (`CPU0: lock(lock_b) ... CPU1: lock(lock_a)`) that would deadlock, from a run where nothing hung.

Exercise — counters nobody shares (~20 min): add a fourth counter to `race.c` that is exact with no lock and no atomic: give each thread its own counter (pass `&mine[cpu]` as `kthread_create()`'s data argument) and add the two up after both threads finish: `race: expected 400000, plain 399961, atomic 400000, spinlock 400000, per-thread 400000`. The kernel counts packets and page faults this way, with per-CPU counters. Answer: `day12/solution/race.c`.

Bonus (5 min): put `msleep(10);` between `rcu_dereference()` and `rcu_read_unlock()` in `rcu_lab`'s reader. The kernel prints `Voluntary context switch within RCU read-side critical section!`, with `msleep` ← `reader_fn` in the trace. The versions still arrive in order, because this kernel's preemptible RCU waits for a reader that blocks; Check (b) says why it's still a bug.

Windows: `atomic_inc` is `InterlockedIncrement`; RCU has no built-in counterpart (the closest is a push lock or an epoch scheme); lockdep resembles Driver Verifier's deadlock detection.

Check: (a) What exactly goes wrong in `READ_ONCE(plain) + 1` on two CPUs? (b) Why may an RCU reader never sleep inside `rcu_read_lock()`?

Day 13 — A PCI driver: registers, interrupts, DMA, hot-plug

Goal: drive QEMU's `edu` card the way real drivers drive hardware.

Read: the one-page edu spec; How To Write Linux PCI Drivers through "Register IRQ handler"; Dynamic DMA mapping Guide: "CPU and DMA addresses" and "Using Coherent DMA mappings".

Build: `cd ~/os/lab/day13 && make`. `edu.c` (207 lines) in four parts:

- **Probe:** `edu_ids` matches vendor 0x1234, device 0x11e8; `pcim_enable_device()` and `pcim_iomap_region()` map BAR 0; all `pcim_`/`devm_` resources free themselves on remove.
- **Registers:** `readl`/`writel` at offsets from the spec (`EDU_FACTORIAL` 0x08, `EDU_IRQ_STATUS` 0x24, `EDU_IRQ_ACK` 0x64).
- **Interrupts:** `pci_set_master()`, `pci_alloc_irq_vectors()` (MSI), then `devm_request_threaded_irq()`. The top half reads the status, acks it and returns `IRQ_WAKE_THREAD`; the thread calls `complete()`.
- **DMA:** `dma_set_mask_and_coherent(dev, DMA_BIT_MASK(28))` (the card sees only 256 MB) and `dmam_alloc_coherent()`; `edu_dma()` writes source, destination, length and command, then sleeps on the completion.

Run:

```
cd /lab/day13; insmod edu.ko         # id 0x010000ed, irq 26, DMA buffer at bus address ...
cd /sys/bus/pci/devices/0000:00:04.0
echo 10 > factorial; cat factorial   # 3628800, computed by the card
echo hello DMA > dma; cat dma        # hello DMA: RAM → card → RAM
grep -w edu /proc/interrupts; ps | grep edu
IRQ=$(grep -w edu /proc/interrupts | cut -d: -f1 | tr -d " ")
echo 1 > /proc/irq/$IRQ/smp_affinity_list    # send its interrupts to CPU 1
for n in 3 4 5 6 7; do echo $n > factorial; done; grep -w edu /proc/interrupts
echo 0000:00:04.0 > /sys/bus/pci/drivers/edu/unbind   # remove() runs
echo 0000:00:04.0 > /sys/bus/pci/drivers/edu/bind     # probe() again
# plug in a second card: Ctrl-a c, then:  device_add edu,id=edu2  then Ctrl-a c
ls /sys/bus/pci/drivers/edu/         # 0000:00:04.0 and 0000:00:06.0
cd /lab; rmmod edu                   # removed, twice
```

Change it: add `#include <linux/delay.h>` and `msleep(10);` at the top of `edu_irq` (the top half), rebuild, reload, `echo 5 > factorial`. Predict first. Then move the line into `edu_irq_thread`, and finally delete it.

Notice:

- The card computed 10! itself, and the DMA text made the round trip without the CPU copying a byte.
- `/proc/interrupts` shows `PCI-MSI ... edu`, and `[irq/26-edu]` is your bottom half. After the affinity change, new interrupts count in the CPU 1 column.
- The hot-plugged card got its own `probe()` and `struct edu`: one driver, many devices.
- `msleep` in the top half: `BUG: scheduling while atomic`, with `edu_irq` in the trace. The machine survives, but lockdep switches itself off after a report like this (`INFO: lockdep is turned off`), so restart the VM afterwards. In the thread it costs 10 ms and nothing else.
- Without `pci_set_master()`, MSI never arrives (an MSI is a memory write by the card), and `echo` fails with `Connection timed out` after one second.

Exercise — is the card alive? (~20 min): register 0x04 in the edu spec is a liveness check: write a value, read back its bitwise inverse. Add a read-write attribute `liveness` (`DEVICE_ATTR_RW`, `kstrtou32()` and `writel()` in the store, `readl()` in the show, listed in `edu_attrs`). `echo 0x12345678 > liveness; cat liveness` prints `0xedcba987`, and after `echo 0 > liveness`, `0xffffffff`. Answer: `day13/solution/edu.c`.

Windows: `edu_ids` is an INF hardware ID; `probe` is `AddDevice` + `IRP_MN_START_DEVICE`; the top half is the ISR and the thread a DPC or work item; `dmam_alloc_coherent` is `AllocateCommonBuffer`.

Check: (a) Why `readl`/`writel` instead of a plain pointer dereference? (b) Why can't you hand the card a `kmalloc` or `vmalloc` pointer for DMA?

Days 14–17: subsystems — block, files, network, USB

Day 14 — The block layer: a RAM disk with a real file system on it

Goal: write a block driver, put ext2 on it, and watch the requests the block layer sends you.

Read: Multi-Queue Block IO Queueing Mechanism (blk-mq): "Introduction" and "Operation"; LKMPG chapter 27, "Block Drivers".

Build: `cd ~/os/lab/day14 && make`. `labdisk.c` (119 lines, modeled on LKMPG's `blkram.c` and the kernel's `brd.c`): `register_blkdev()`, a `blk_mq_tag_set` with one hardware queue, `blk_mq_alloc_disk()`, `add_disk()`. The block layer calls `labdisk_queue_rq()` with each request:

```
    blk_mq_start_request(rq);
    rq_for_each_segment(bvec, rq, iter) {   /* each piece of each page */
        void *buf = bvec_kmap_local(&bvec);

        if (rq_data_dir(rq) == WRITE)
            memcpy(data + pos, buf, bvec.bv_len);
        else
            memcpy(buf, data + pos, bvec.bv_len);
        kunmap_local(buf);
        pos += bvec.bv_len;
    }
    blk_mq_end_request(rq, BLK_STS_OK);
```

Run:

```
cd /lab/day14; insmod labdisk.ko
ls -l /dev/labdisk; cat /sys/block/labdisk/size          # b 251,0; 32768 sectors
cd /sys/block/labdisk/queue; cat logical_block_size max_sectors_kb scheduler; cd /lab/day14
mke2fs /dev/labdisk > /dev/null; mount -t ext2 /dev/labdisk /mnt
cd /sys/kernel/tracing
echo 1 > events/block/block_rq_issue/enable; echo 1 > events/block/block_rq_complete/enable
echo hello > /mnt/hello.txt; sync
echo 0 > events/block/block_rq_issue/enable; echo 0 > events/block/block_rq_complete/enable
grep -E "block_rq_(issue|complete)" trace | tail -4
cd /; umount /mnt; rmmod labdisk
```

Notice:

- The kernel ran a real file system on 119 lines of yours: `EXT4-fs (labdisk): mounting ext2 file system using the ext4 subsystem`.
- The queue chose `[mq-deadline]` as the I/O scheduler and caps requests at 127 KB.
- Writing one file then `sync` caused 7 requests, issued by a `kworker` (writeback), not by your shell: `block_rq_issue: 251,0 WM 1024 () 522 + 2` means write (W) of metadata (M), 1024 bytes, starting at sector 522 for 2 sectors.

Exercise — a slow disk (~30 min): add a `delay_ms` parameter (mode 0644, default 10) and `msleep(delay_ms)` at the start of `labdisk_queue_rq()`. Load it and read the kernel's complaint: `Voluntary context switch within RCU read-side critical section!`, with `msleep` ← `labdisk_queue_rq` in the trace. blk-mq calls `queue_rq` inside an RCU read-side section unless you ask otherwise. So set `tag_set.flags = BLK_MQ_F_BLOCKING;` before `blk_mq_alloc_tag_set()`, reboot the VM (that warning only fires once per boot) and load it again: no warning, and `time dd if=/dev/zero of=/dev/labdisk bs=4k count=50 oflag=direct` takes 0.67 s, against 0.06 s after `echo 0 > /sys/module/labdisk/parameters/delay_ms`. Answer: `day14/solution/labdisk.c`.

Windows: a storage miniport, roughly: the block layer plays the storport/class driver role, and `queue_rq` is where SRBs would arrive.

Check: (a) Why did `echo hello` not cause a block request at once? (b) What does the I/O scheduler do, and why do NVMe disks usually use `none`?

Day 15 — The VFS: your own file system

Goal: register a file system, mount it, and follow `read()` from user space into your function through the VFS.

Read: Overview of the Linux Virtual File System: "Introduction" and "Registering and Mounting a Filesystem"; Linux Kernel Labs File system drivers (Part 1): "Overview".

Build: `cd ~/os/lab/day15 && make`. `labfs.c` (110 lines, modeled on securityfs) describes its whole tree in a table and lets libfs do the rest:

```
    static const struct tree_descr files[] = {
        [2] = { "hello",   &hello_fops,   0444 },   /* index = inode number */
        [3] = { "jiffies", &jiffies_fops, 0444 },
        [4] = { "note",    &note_fops,    0644 },
        { "" }
    };
    return simple_fill_super(sb, LABFS_MAGIC, files);
```

Mounting goes through the 7.x `fs_context` API: `init_fs_context` sets `get_tree_single()`, which calls the fill function once.

Run:

```
cd /lab/day15; insmod labfs.ko
grep labfs /proc/filesystems             # nodev labfs
mount -t labfs none /mnt; ls -li /mnt    # inodes 2, 3, 4
cat /mnt/hello; cat /mnt/jiffies; cat /mnt/jiffies
echo "my note" > /mnt/note; cat /mnt/note
cd /sys/kernel/tracing
echo hello_read > set_ftrace_filter; echo 1 > options/func_stack_trace
echo function > current_tracer
cat /mnt/hello > /dev/null
cat trace | grep -v "^#" | head -8       # read it before switching tracers: that clears the
buffer
echo nop > current_tracer; echo 0 > options/func_stack_trace; echo > set_ftrace_filter
cd /; umount /mnt; rmmod labfs
```

Notice:

- `nodev` means no block device behind it, like `/proc`. Inode numbers are your table indexes.
- `jiffies` gives a new number on every `cat`: like Day 1's `/proc` files, the content is made up when read.
- The stack: `hello_read` ← `vfs_read` ← `ksys_read` ← `do_syscall_64` ← `entry_SYSCALL_64_after_hwframe`. The VFS turned a generic `read()` into a call through your `file_operations`.

Exercise — who's reading? (~20 min): add a fifth file, `whoami` (inode 5, mode 0444), whose `read()` reports `current→pid` and `current→comm` with `scnprintf()` and `simple_read_from_buffer()`. `cat /mnt/whoami` prints `pid 109 (cat)`, and `head -c 40 /mnt/whoami` prints `pid 110 (head)`. `read()` runs in the calling process, so `current` is the reader: the same reason Day 11's `read()` could sleep. Answer: `day15/solution/labfs.c`.

Windows: a file system driver registers with the I/O manager and receives `IRP_MJ_READ`; here the VFS dispatches through `file_operations`.

Check: (a) What are the inode, the dentry and the file, and which one holds your read position? (b) Why can't `umount` succeed while a shell has its current directory inside `/mnt`?

Day 16 — Networking: a driver that answers ping

Goal: write a network driver whose pretend host answers ping, then filter packets with netfilter.

Read: Network Devices, the Kernel, and You!: "Introduction" through "MTU"; struct sk_buff.

Build: `cd ~/os/lab/day16 && make`.

- `labnet.c` (modeled on `drivers/net/dummy.c` and LKMPG's `vnetloop.c`): `alloc_netdev("lab%d")` + `ether_setup`, `IFF_NOARP`, and `ndo_start_xmit`. For an ICMP echo request it makes a reply with `skb_copy()`, swaps the IP addresses, sets `ICMP_ECHOREPLY`, recomputes the ICMP checksum, and hands it to `netif_rx()` as if it came off the wire.
- `labfw.c`: `nf_register_net_hook()` at `NF_INET_LOCAL_OUT`; logs each outgoing ping and returns `NF_DROP` when its `drop` parameter is set.

Run:

```
cd /lab/day16; insmod labnet.ko
ip link set lab0 up; ip addr add 10.9.0.1/24 dev lab0
ping -c 3 10.9.0.2                          # answered by your driver
cat /sys/class/net/lab0/statistics/tx_packets /sys/class/net/lab0/statistics/rx_packets
insmod labfw.ko; ping -c 1 10.9.0.2         # labfw: ping #1 to 10.9.0.2 accepted
echo Y > /sys/module/labfw/parameters/drop
ping -c 1 -W 2 10.9.0.2                     # ping: sendto: Operation not permitted
rmmod labfw labnet
```

Notice:

- `64 bytes from 10.9.0.2: seq=0 ttl=64`: routing chose `lab0`, the stack built the packet, your driver turned it around, and the IP and ICMP layers delivered it to `ping`.
- Each ping passes `xmit 98 bytes, ethertype 0x0800`: 14 Ethernet + 20 IP + 8 ICMP + 56 data. The first packet out, 70 bytes of `0x86dd`, was IPv6 starting up.
- A netfilter `NF_DROP` at `LOCAL_OUT` shows up in the sending program as `EPERM`.

Exercise — lose packets quietly (~20 min): move `labfw`'s hook to `NF_INET_LOCAL_IN` and drop every second ICMP packet it sees. `ping -c 4 10.9.0.2` now ends with `4 packets transmitted, 2 packets received, 50% packet loss`: replies 0 and 2 arrive, and nothing fails in `sendto`, because the packets are dropped on the way in, after your driver answered. At `LOCAL_OUT` the sender got `EPERM` instead. Answer: `day16/solution/labfw.c`.

Windows: an NDIS miniport (`MiniportSendNetBufferLists`, `NdisMIndicateReceiveNetBufferLists`); netfilter hooks are what WFP callouts do.

Check: (a) Why does `xmit` use `GFP_ATOMIC` for its copy? (b) Swapping the source and destination addresses keeps the IP header checksum valid. Why?

Day 17 — USB: build a device the system doesn't recognize, then drive it

Goal: assemble a vendor-specific USB device from the shell, watch Linux enumerate it and find no driver, then write that driver.

Read: Writing USB Device Drivers; Linux USB gadget configured through configfs, sections 1–4.

Build: `cd ~/os/lab/day17 && make`. The VM has a software USB host controller wired to a software device controller (`dummy_hcd`), so the device side and the host side run in the same kernel.

- `gadget.sh` builds the device in `/sys/kernel/config/usb_gadget`: vendor `0xcafe`, product `0x0001`, strings, one configuration with the kernel's `Loopback` function (interface class 0xff, two bulk endpoints), then `echo dummy_udc.0 > UDC` plugs it in.
- `usb_echo.c` is the host driver: `USB_DEVICE(0xcafe, 0x0001)` in its ID table; `probe()` finds the bulk endpoints with `usb_find_common_endpoints()`, sends "hello device" with `usb_bulk_msg()` and reads the echo. Buffers come from `kmalloc`, never the stack: USB may DMA from them.

Run:

```
cd /lab/day17; sh gadget.sh             # usb 1-1: New USB device found, idVendor=cafe ...
I=/sys/bus/usb/devices/1-1:1.0           # the device's first interface
cat $I/bInterfaceClass $I/bNumEndpoints $I/modalias; ls $I/driver   # ff, 02, ..., no driver
ls -d $I/ep_*                            # ep_02 (OUT), ep_81 (IN)
insmod usb_echo.ko                       # sent 13 bytes, device echoed 13: hello device
ls -l $I/driver                          # → usb_echo
echo > /sys/kernel/config/usb_gadget/lab/UDC   # unplug: USB disconnect ... unplugged
rmmod usb_echo
```

Notice:

- The host read the descriptors you wrote (`Manufacturer: Linux Internals Lab`, `Product: Echo device`), and with interface class 0xff no driver claimed it, until your `USB_DEVICE(0xcafe, 0x0001)` matched the modalias `usb:vCAFEp0001...`.
- Unplugging called `disconnect()` without `rmmod`, the way a real cable pull does.
- `usb_echo.ko` works unchanged against a real board with the same IDs: see the ESP32-S3 in After Day 22.

Exercise — send on demand (~40 min): keep what `probe()` found (the `usb_device`, both endpoint addresses, a 64-byte `devm_kmalloc()` buffer and a mutex) in a struct attached with `usb_set_intfdata()`, move the round trip into a function, and add a write-only attribute `send` through `usb_driver`'s `.dev_groups`. `echo hi > /sys/bus/usb/devices/1-1:1.0/send` logs `usb_echo 1-1:1.0: sent 2 bytes, device echoed 2: hi`. The driver core creates the file after `probe()` and removes it before `disconnect()`, so a write can't race with unplugging. Answer: `day17/solution/usb_echo.c`.

Windows: the IDs match like an INF; with no INF, Windows shows an unknown device, the same state you saw before `insmod`; `usb_bulk_msg` is a synchronous URB, as with WinUSB.

Check: (a) Why may `probe()` call `usb_bulk_msg()`, which sleeps, while `disconnect()` must assume the device is already gone? (b) What would change if the gadget used the `acm` function instead of `Loopback`?

Day 18: debug like a kernel developer, then send a patch

Day 18 — KASAN, kmemleak, gdb, and your first patch

Goal: the four tools you'll use on every real driver bug, and the path a change takes into the kernel.

Read: KASAN: "Overview", "Usage", "Error reports"; kmemleak: "Usage"; Debugging kernel and modules via gdb: "Setup" and "Examples"; Submitting patches: "Describe your changes" through "Select the recipients for your patch", and "Sign your work".

Build: `cd ~/os/lab/day18 && make`. `buggy.c` writes after `kfree` (`bug=1`) or one byte past a 16-byte `kmalloc` (`bug=2`). `leak.c` allocates four 16 KB buffers in a loop and keeps none.

Run 1, memory bugs: boot with two extra kernel options. By default KASAN reports only the first bug per boot, and kmemleak stays off because it slows everything down.

```
~/os/lab/setup/run.sh kasan_multi_shot kmemleak=on
cd /lab/day18
insmod buggy.ko bug=1; rmmod buggy      # use after free
insmod buggy.ko bug=2; rmmod buggy      # out of bounds
insmod leak.ko; sleep 6
echo scan > /sys/kernel/debug/kmemleak  # a leak must look unreferenced in two scans
echo scan > /sys/kernel/debug/kmemleak  # kmemleak: 4 new suspected memory leaks
cat /sys/kernel/debug/kmemleak | head -16
```

Run 2, gdb on the live kernel: two terminals. Once: `echo "add-auto-load-safe-path $HOME/os/linux" >> ~/.gdbinit`.

```
# terminal 1
~/os/lab/setup/run.sh
insmod /lab/day11/kbuf.ko
# terminal 2
cd ~/os/linux && gdb vmlinux
(gdb) target remote :1234          # the whole VM freezes
(gdb) lx-symbols ~/os/lab/day11    # load symbols for modules loaded right now
(gdb) break kbuf_write
(gdb) continue
# terminal 1: echo hi > /dev/kbuf  (it hangs at your breakpoint)
(gdb) bt
(gdb) p count
(gdb) p $lx_current().comm
(gdb) x/s buf                      # user memory of the calling process
(gdb) next
(gdb) delete
(gdb) continue
```

Run 3, the patch: turn Day 7's system call into a patch, as you would any kernel change:

```
cd ~/os/linux
git config --global user.name "Your Name"; git config --global user.email "you@example.com"
git switch -c hello-syscall
git commit -a -s -m "x86/syscalls: add hello() for learning" \
    -m "Adds a toy system call that logs its caller and returns twice its argument. Written
while learning; not meant for merging."
git format-patch -1 -o ~/os/patches
./scripts/checkpatch.pl ~/os/patches/0001-*.patch
./scripts/get_maintainer.pl ~/os/patches/0001-*.patch
./scripts/checkpatch.pl -f ~/os/lab/day06/list.c ~/os/lab/day13/edu.c
```

Don't send it. A real patch goes out with `git send-email` to the addresses `get_maintainer.pl` printed.

Notice:

- KASAN: `BUG: KASAN: slab-use-after-free in buggy_init+0xe5`, `Write of size 1`, then where the object was allocated and where it was freed. For `bug=2`: `slab-out-of-bounds ... 0 bytes to the right of allocated 16-byte region`.
- kmemleak: `unreferenced object ... (size 16384)`, a hex dump showing `lost buffer 0`, and the allocating backtrace. Your module's init code appears as a bare address: init code is freed after loading. kmemleak is conservative: any stale value that looks like a pointer keeps an object alive, so small slab objects can slip through; these buffers come straight from the page allocator.
- gdb: the backtrace is the system call path, `entry_SYSCALL_64` → `do_syscall_64` → `ksys_write` → `vfs_write` → `kbuf_write`, and `x/s buf` prints `"hi\n\001"` straight from user memory: the three bytes you wrote, then whatever byte comes next, because `write()` data has no terminating zero. `p $lx_current().comm` shows `"sh"`, the shell that ran `echo`. Everything stops while you're at the breakpoint; afterwards the kernel may grumble `clocksource: Watchdog ... read timed out`.
- checkpatch flags the commit message (`Prefer a maximum 75 chars per line`), suggests `kmalloc_obj(*r, GFP_KERNEL)` in `list.c` (a 7.x helper that sizes the allocation from the type, like C++ `new`), warns that `%px` exposes kernel addresses, and wants a newline at the end of `dma_show`'s format. Judge each warning: here the `%px` is deliberate.
- `get_maintainer.pl` names the x86 entry-code and x86 architecture maintainers, `x86@kernel.org` and `linux-kernel@vger.kernel.org`. `-s` added `Signed-off-by:`, your statement under the Developer's Certificate of Origin.

Exercise — the mystery module (~30 min): `day18/mystery.c` loads, prints `mystery: sum of the first 1000 squares is 332833500`, and hides two bugs. Find them with the tools, not by reading: in a VM booted with `kasan_multi_shot kmemleak=on`, `insmod /lab/day18/mystery.ko`, read the KASAN report, `rmmod mystery`, then run two kmemleak scans. Turn the report's address into a source line on your machine with `cd ~/os/linux && ./scripts/faddr2line ~/os/lab/day18/mystery.ko mystery_init+0x152`, using the offset your report shows but without its `/size` part (a loaded module's init code reports a different size than the file holds). You should get `slab-out-of-bounds`, `Write of size 4`, `0 bytes to the right of allocated 4000-byte region` and `mystery.c:29`, plus one unreferenced object of size 16384 whose hex dump begins `sum of the first`. Fix both, and check that the fixed module is silent. Answer: `day18/solution/mystery.c`.

Windows: KASAN is Driver Verifier's Special Pool; kmemleak resembles Pool Tracking (`!poolused`); gdb over QEMU's stub is WinDbg over KDNET.

Check: (a) Why don't distributions ship kernels with KASAN or kmemleak on? (b) What are mainline, stable, longterm and linux-next, and which one should a new patch be based on?

Days 19–22: time, containers, an ARM board, eBPF

Four more parts of the job that interviews and real work keep coming back to. Days 19 and 20 run in the x86 VM, as before. Day 21 builds and boots an arm64 kernel. Day 22 runs on your own machine's kernel, which is safe because the kernel checks every eBPF program before running it.

Day 19 — Deferred work and time: timers, hrtimers, workqueues

Goal: run one periodic job three ways, see which context each one runs in, then break the rule that decides which one you pick.

Read: Workqueue: "Introduction", "The Design" and "Guidelines"; Delay and sleep mechanisms, all of it; the first section of hrtimers, on why the kernel has two kinds of timer.

Build: `cd ~/os/lab/day19 && make`. `deferred.c` (88 lines) starts three things when it loads. Each callback prints its context (`in_hardirq()`, `in_serving_softirq()`, `in_task()`), its CPU and `current->comm`:

- a `timer_list` (`timer_setup()`, `mod_timer()`), due every 100 ms, three times. It counts in jiffies, 1 ms each here (`CONFIG_HZ=1000`). It may not sleep, so it hands the slow part to a work item with `schedule_work()`;
- that work item (`INIT_WORK()`), which runs in a kernel thread and may sleep: it calls `msleep(1)`;
- an `hrtimer` (`hrtimer_setup()`, `hrtimer_start()`), due every 500 µs, 1000 times. It re-arms itself with `hrtimer_forward_now()` and `HRTIMER_RESTART`, and records the shortest and longest gap between runs.

Unloading calls `timer_shutdown_sync()`, `hrtimer_cancel()` and `cancel_work_sync()`. Each waits for a running callback to finish, so none of the module's code runs after it's gone.

Run:

```
cd /lab/day19
grep -E "^ *(TIMER|HRTIMER):" /proc/softirqs; grep LOC: /proc/interrupts
insmod deferred.ko; sleep 1; rmmod deferred
dmesg | grep deferred
grep -E "^ *(TIMER|HRTIMER):" /proc/softirqs; grep LOC: /proc/interrupts
```

Change it: add `msleep(1);` to `timer_fn`, right after `schedule_work()`. Rebuild and reload. Predict first.

Notice:

- `deferred: hrtimer hardirq=1 softirq=0 task=0 cpu=1 current=insmod`: the hrtimer callback ran inside a hard interrupt, on top of whatever that CPU was running: here the `insmod` that had just armed it. In interrupt context, `current` is only the task that happened to be interrupted.
- `timer hardirq=0 softirq=1 task=0 cpu=1 current=sh`, later `current=swapper/0`: `timer_list` callbacks run in the TIMER softirq, on the way out of an interrupt, again on top of whatever ran there. `swapper/0` is CPU 0's idle task.
- `work hardirq=0 softirq=0 task=1 cpu=1 current=kworker/1:2`: the work item ran in process context, in a worker thread of CPU 1's pool, so its `msleep(1)` is allowed.
- `hrtimer ran 1000 times; asked for 500 us, got gaps of 44..8795 us`. That's emulation; under KVM the gaps stay close to 500 µs. A `timer_list` can only expire on a jiffy, which suits timeouts that rarely fire; an hrtimer programs the timer hardware for its exact expiry.

- TIMER went from 618 to 673 on CPU 0, but HRTIMER stayed at 0: hrtimers expire straight from the timer interrupt, not from a softirq, unless you start them in a `_SOFT` mode. They count in `LOC`, the local timer interrupts, together with the scheduler tick.
- With `msleep` in the timer: `BUG: scheduling while atomic: swapper/1/0/0x00000102`, with `msleep` ← `timer_fn [deferred]` ← `call_timer_fn` inside `<IRQ>` in the trace, and `INFO: lockdep is turned off`. The last number is the preempt count; its 0x100 bit means "serving a softirq". The VM survives; restart it before you go on.

Exercise — a timer that may sleep (~20 min): replace the `timer_list` and the work item with one `struct delayed_work`: set it up with `INIT_DELAYED_WORK()`, start and re-arm it with `schedule_delayed_work(&tick, msecs_to_jiffies(100))`, have it `msleep(10)` on every run, and unload with `cancel_delayed_work_sync()`. You should see `deferred: tick hardirq=0 softirq=0 task=1 cpu=0 current=kworker/0:2` three times, about 120 ms apart, because each run re-arms only after its sleep. This is how drivers poll hardware that has no interrupt. Answer: `day19/solution/deferred.c`.

Windows: a `timer_list` is a `KTIMER` with a DPC, and both run where you can't wait (`DISPATCH_LEVEL`); a work item is an `IoQueueWorkItem`; an hrtimer is a high-resolution timer (`ExAllocateTimer` with `EX_TIMER_HIGH_RESOLUTION`).

Check: (a) Tasklets are another bottom half you'll find in older drivers. Why do new drivers avoid them, and what do they use instead? (b) What would happen if `deferred_exit` skipped `cancel_work_sync()`, and why does it shut the timer down first?

Day 20 — Namespaces and cgroups: a container from scratch

Goal: take a container apart. Give a shell its own hostname, PIDs, network and mounts, cap what it may use, then write the runtime yourself.

Read: namespaces(7) through "The /proc/pid/ns/ directory"; Control Group v2: "Introduction", "Basic Operations" and "Resource Distribution Models"; LWN's Namespaces in operation, part 1.

Build: nothing yet: BusyBox's `unshare` does the first half. A namespace gives a process its own copy of one kind of global name (hostname, PIDs, mounts, network devices). A cgroup puts processes in a group whose resources a controller limits and counts.

Run:

```
# namespaces: each unshare runs its command in new ones
hostname; unshare -u sh -c "hostname box; hostname"; hostname
unshare -p -f --mount-proc sh -c 'echo $$; ps'
unshare -n ip link
ls -l /proc/$$/ns
unshare -m sh -c "mount -t tmpfs none /mnt; touch /mnt/inside; ls /mnt"; ls /mnt; echo
outside-done
# cgroups: limits for a group of processes
mount -t cgroup2 none /sys/fs/cgroup; cat /sys/fs/cgroup/cgroup.controllers
echo "+pids +cpu" > /sys/fs/cgroup/cgroup.subtree_control
mkdir /sys/fs/cgroup/box; echo 3 > /sys/fs/cgroup/box/pids.max
sh -c 'echo $$ > /sys/fs/cgroup/box/cgroup.procs; sleep 2 & sleep 2 & sleep 2 & wait'
cat /sys/fs/cgroup/box/pids.events
# two CPU hogs on CPU 1, in groups weighted 100 and 900: keep the last line on one line
cd /sys/fs/cgroup; mkdir a b; echo 100 > a/cpu.weight; echo 900 > b/cpu.weight; cd /lab
sh -c 'echo $$ > /sys/fs/cgroup/a/cgroup.procs; exec taskset -c 1 yes > /dev/null' & A=$!; sh
-c 'echo $$ > /sys/fs/cgroup/b/cgroup.procs; exec taskset -c 1 yes > /dev/null' & B=$!; sleep
5; grep usage_usec /sys/fs/cgroup/a/cpu.stat /sys/fs/cgroup/b/cpu.stat; kill $A $B
```

Notice:

- `(none)`, `box`, `(none)`: the new UTS namespace got its own hostname, and yours never changed.
- In the new PID namespace the shell is PID `1`, and `ps` shows nothing but itself: `--mount-proc` mounted a `/proc` that lists only that namespace. Outside, the same process has an ordinary PID.
- The new network namespace has only `lo`, and it's down: no interface leads out until someone moves one end of a veth pair in.
- `/proc/$$/ns` holds one link per namespace type, such as `uts → uts:[4026531838]`. Two processes share a namespace exactly when the numbers match. There's no `user` link: this kernel is built without `CONFIG_USER_NS`. Ubuntu's has it, and that's what lets `unshare -U -r` make an ordinary user root inside a container.
- The tmpfs mounted in the new mount namespace never appeared outside: `inside`, then an empty `ls`.
- The controllers are `cpuset cpu io hugetlb pids rdma misc`. There's no `memory`: this kernel has no `CONFIG_MEMCG`. On Ubuntu, `memory.max` is the limit that makes the OOM killer pick a process in the container instead of anywhere on the machine.
- `sh: can't fork: Resource temporarily unavailable` for the third `sleep`, since the shell counts as one of the three, and `pids.events` reads `max 1`. A fork bomb in a container stops there.
- `a/cpu.stat:usage_usec 509095` against `b/cpu.stat:usage_usec 4503129`: 1 to 9, the ratio of the weights. It's Day 8's proportional share, applied to groups instead of tasks; 100 is the default weight.

Exercise — box, a container in 42 lines (~40 min): write `box.cpp`. It `clone()`s a child with `CLONE_NEWUTS | CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET | SIGCHLD` on a 1 MB stack of its own (`clone()` takes the stack's top address). The child sets the hostname to `box` and makes its mounts private, with `mount(nullptr, "/", nullptr, MS_REC | MS_PRIVATE, nullptr)`, so its new `/proc`

mount stays inside. It joins `/sys/fs/cgroup/box` by writing `0` ("me") to its `cgroup.procs`, then runs `execl("/bin/sh", "sh", nullptr)`. The parent prints the child's PID and waits. Build it with `g++ -std=c++14 -static` and run it in the VM after the steps above. You should see `box: started pid 114 on the outside`; inside, `echo $$` prints `1`, `hostname` prints `box`, `ps` lists only `sh` and `ps`, `ip link` only `lo`, and `sleep 10 & sleep 10 & sleep 10 &` can't fork the third. After `exit`, the VM's hostname is still `(none)`. Answer: `day20/solution/box.cpp`. Docker, Podman and Kubernetes make the same system calls through `runc`, adding a root file system of its own (`pivot_root`), a user namespace, seccomp filters and dropped capabilities.

Windows: Windows containers run in server silos, built on job objects (the closest thing to a cgroup) with their own view of the object namespace; Hyper-V isolation runs each container in a small VM instead.

Check: (a) A container and a virtual machine both isolate programs. What does each share with the host, and what does that mean for security? (b) Why does `unshare -p` need `-f`?

Day 21 — Embedded: a device tree and a platform driver on arm64

Goal: cross-compile Linux for an ARM board, describe a device the hardware can't announce, and write the driver that binds to that description.

Read: Linux and the Devicetree, sections 1 and 2; Platform Devices and Drivers, the first three sections; GPIO Descriptor Consumer Interface, "Guidelines for GPIOs consumers" through "Spinlock-Safe GPIO Access".

Build: a second kernel, for QEMU's `virt` board, from the same repository. Then QEMU's own description of that board, plus one device:

```
sudo apt install -y gcc-aarch64-linux-gnu qemu-system-arm device-tree-compiler
~/os/lab/setup/build-arm64.sh          # 13 minutes on the test machine's 4 cores
cd ~/os/lab/day21
QEMU_EXTRA="-M dumpdtb=$PWD/virt.dtb" ~/os/lab/setup/run-arm64.sh
dtc -q -I dtb -O dts -o virt.dts virt.dtb
grep -A9 "pl061@" virt.dts; grep -A4 "poweroff {" virt.dts
dtc -q -I dts -O dtb -o lab.dtb lab.dts
make                                   # cross-compiles blinker.ko for arm64
```

- `build-arm64.sh` adds a worktree at v7.3, configures it with `defconfig virt.config mod2noconfig` (a virtual machine's drivers, every optional module dropped) plus built-in 9p and the function tracer, and builds it with `ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu-`. Then it fetches Ubuntu's arm64 BusyBox and packs the same `init` as the x86 VM. `run-arm64.sh` boots it with `-M virt -cpu cortex-a57`, and `/lab` is shared as before.
- `lab.dts` includes the decompiled `virt.dts` and adds one node: `compatible = "lab,blinker"`, `led-gpios = <&{/pl061@9030000} 4 0>` (line 4 of the board's PL061 GPIO controller, active high) and `lab,period-ms = <500>`. A property that isn't a standard binding carries a vendor prefix, and its unit goes in its name.

- `blinker.c` (75 lines) is a `platform_driver` whose `of_match_table` lists `lab,blinker`. `probe()` takes the line with `devm_gpiod_get(dev, "led", GPIOD_OUT_LOW)`, which reads the `led-gpios` property, and the period with `device_property_read_u32()`. A `delayed_work` flips the line with `gpiod_set_value()`. `remove()` cancels the work, and `devm` gives back the line and the memory.

Run: two terminals on your machine. `-trace pl061_set_output` makes QEMU log every change on the chip's output pins: the hardware's side of your driver.

```
# terminal 1
DTB=$PWD/lab.dtb QEMU_EXTRA="-trace pl061_set_output -D $PWD/pins.log" ~/os/lab/setup/run-
arm64.sh
cat /proc/device-tree/blinker/compatible; echo; od -An -tx1 /proc/device-
tree/blinker/lab,period-ms
ls /sys/bus/platform/devices/blinker/; cat /sys/bus/platform/devices/blinker/modalias
cat /sys/kernel/debug/gpio
insmod /lab/day21/blinker.ko; cat /sys/kernel/debug/gpio
cd /sys/kernel/tracing; echo pl061_set_value > set_ftrace_filter; echo 1 >
options/func_stack_trace
echo function > current_tracer; sleep 1; grep -v "^#" trace | head -12
echo 0 > options/func_stack_trace; echo nop > current_tracer; cd /lab
echo blinker > /sys/bus/platform/drivers/blinker/unbind; cat /sys/kernel/debug/gpio
echo blinker > /sys/bus/platform/drivers/blinker/bind; rmmod blinker
# terminal 2, while blinker is loaded
tail -f ~/os/lab/day21/pins.log
```

Change it: delete the `led-gpios` line from `lab.dts`, compile it again with `dtc`, boot with it, and `insmod` the module. Then put the line back.

Notice:

- In `virt.dts`, the controller is `pl061@9030000` with `phandle = <0x8005>`, `reg = <0x00 0x9030000 0x00 0x1000>` and `compatible = "arm,pl061\0arm,primecell"`, and the `gpio-keys` node points at it with `gpios = <0x8005 0x03 0x00>`: line 3 is QEMU's power button. After `dtc`, your node reads `led-gpios = <0x8005 0x04 0x00>`: the path reference became that phandle.
- The arm64 VM boots in 6 seconds, emulated, because this kernel has no optional drivers.
- `od` prints `00 00 01 f4`: 500 as one big-endian 32-bit cell, the device tree's only number format.
- The device existed before its driver did: `/sys/bus/platform/devices/blinker` has a `modalias`, `of:NblinkerT(null)Clab,blinker`, which udev would use to load a module that matches it, and a `supplier:amba:9030000.pl061` link. The kernel read `led-gpios` and added a device link, so `blinker` can't probe before its GPIO controller.
- `/sys/kernel/debug/gpio` lists `gpio-3 ( |GPIO Key Poweroff ) in lo IRQ`; after `insmod`, also `gpio-4 ( |led ) out hi`, named after the `"led"` you passed. dmesg says `blinker blinker: blinking every 500 ms`.
- The trace shows a kworker calling `pl061_set_value` twice a second, from `blink` → `gpiod_set_value` → `gpiod_set_value_nocheck` → `gpiod_set_raw_value_commit` → `gpiochip_set` → `pl061_set_value`: your driver, then gpiolib, then the controller's driver.

- `pins.log` grows by `pl061_set_output /machine/unattached/device[8] setting output 4 to 1`, then `to 0`, ten lines in 5 seconds: QEMU's model of the chip received the register writes. After `unbind`, only `gpio-3` is left and the log stops growing.
- Without `led-gpios`: `blinker blinker: error -ENOENT: no led-gpios`, then `probe with driver blinker failed with error -2`. `dev_err_probe()` wrote the first line; the driver core wrote the second.

Exercise — a knob in sysfs (~25 min): add a read-write attribute `period_ms` that shows and changes the period while the driver runs: `DEVICE_ATTR_RW(period_ms)`, `sysfs_emit()` in the show, `kstrtou32()` in the store, refusing 0 with `-EINVAL`. Register it through the driver's `.dev_groups` (`ATTRIBUTE_GROUPS(blinker)`), so the driver core creates the file after `probe()` and removes it before `remove()`. `blink()` now reads a value another thread may write, so use `READ_ONCE()` and `WRITE_ONCE()` (Day 12). `cat /sys/bus/platform/devices/blinker/period_ms` prints `500`; after `echo 100 > period_ms`, `pins.log` grows by about 45 lines in 5 seconds instead of 10, and `echo 0 > period_ms` fails with `write error: Invalid argument`. Answer: `day21/solution/blinker.c`.

On a real board you wouldn't write this driver: `compatible = "gpio-leds"` binds `drivers/leds/leds-gpio.c`, which puts the LED in `/sys/class/leds` with blink triggers. Before writing a driver, look for a generic binding in `Documentation/devicetree/bindings/`.

Windows: an ARM PC running Windows describes its hardware with ACPI tables, not a device tree. Linux reads ACPI too, on x86 and on ARM servers, and `device_property_read_u32()` reads the property from either. A platform driver is like a KMDF driver for an ACPI-enumerated device, and GPIO lines reach it through GpioClx as connection resources.

Check: (a) Why does an ARM board need a device tree when a PC doesn't? (b) `blinker` called `device_property_read_u32()`, not `of_property_read_u32()`. What's the difference?

Day 22 — eBPF: programs that run inside the kernel

Goal: trace your own machine's kernel with bpftrace, then write the two halves of an eBPF tool: a program the kernel verifies and runs, and a C++ loader that reads its results.

Read: libbpf Overview: "BPF App Lifecycle and libbpf APIs" and "BPF CO-RE"; eBPF verifier, its introduction and "Understanding eBPF verifier messages"; the BPF_MAP_TYPE_HASH page.

Build: on your machine, not in the VM: `sudo apt install -y bpftrace clang libbpf-dev`, then `cd ~/os/lab/day22 && make`.

- `syscalls.bpf.c` (35 lines) is the part that runs in the kernel. clang compiles it to BPF bytecode (`-target bpf`; `-g` adds BTF, the type information libbpf needs to create maps). `SEC("tracepoint/raw_syscalls/sys_enter")` attaches it to every system call entry. It reads the caller's name with `bpf_get_current_comm()` and counts it in `counts`, a hash map of up to 1024 entries that the kernel keeps: `bpf_map_lookup_elem()`, then `__sync_fetch_and_add()` because other CPUs count at the same time, or `bpf_map_update_elem()` for a new name.
- `syscalls.cpp` (51 lines) is the loader. `bpf_object__open_file()` reads the object file. `bpf_object__load()` creates the map and loads the program; this is where the verifier runs. `bpf_program__attach()` hooks it to the tracepoint. After the sleep it walks the map with

`bpf_map_get_next_key()` and `bpf_map_lookup_elem()`, which are wrappers around the `bpf()` system call, and prints the top ten.
- `getppid.bt` and `runqlat.bt` are bpftrace scripts. bpftrace compiles its own language to the same bytecode, with maps written as `@name`.

Run:

```
cd ~/os/lab/day22
# every file anybody opens; Ctrl-C to stop
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_openat { printf("%-16s %s\n", comm,
str(args.filename)); }'
# Day 1's getppid, timed inside the kernel: start it, run ../day01/syscall_cost in a second
terminal, then Ctrl-C
sudo bpftrace getppid.bt
# the kernel stack at a tracepoint: start it, run dd if=/etc/os-release of=/dev/null in a
second terminal, Ctrl-C
sudo bpftrace -e 'tracepoint:syscalls:sys_enter_read /comm == "dd"/ { @[kstack] = count(); }'
# how long woken tasks wait for a CPU: 5 s idle, then 5 s with two busy loops per CPU
sudo timeout -s INT 5 bpftrace runqlat.bt
for i in $(seq $((2 * $(nproc)))); do timeout 7 sh -c 'while :; do :; done' & done; sleep 1;
sudo timeout -s INT 5 bpftrace runqlat.bt
# your own eBPF program: count for 5 s while dd makes 400,000 system calls
(sleep 1; dd if=/dev/zero of=/dev/null bs=1 count=200000) & sudo ./syscalls 5
```

Change it: in `syscalls.bpf.c`, replace the `if`/`else` with a bare `__sync_fetch_and_add(count, 1);`, as if the name were always in the map already. `make` and run `sudo ./syscalls` again.

Notice:

- The open trace shows every program on the machine, live; on the test machine it was a compiler opening headers.
- Inside the kernel, most `getppid` calls took 64 to 256 ns (`@ns` histogram buckets `[64, 128)` and `[128, 256)`), but `syscall_cost` reported over 400 ns per call while traced, against 181 ns without. Two probes per system call cost more than the call itself: tracing is cheap, not free.
- The stack is Day 7's path: `entry_SYSCALL_64_after_hwframe` → `do_syscall_64` → `syscall_trace_enter` → `perf_syscall_enter`. The top frames are the tracepoint's own.
- Run-queue latency: idle, every wakeup got a CPU within 128 µs. With 8 busy loops on 4 CPUs, a tail reached 1 to 4 ms: woken tasks waited for a time slice to end, Day 8's scheduler at work.
- `syscalls` printed `400048 dd`: 200,000 reads and 200,000 writes, plus 48 for starting up. Names are cut at 15 characters, because `comm` in `task_struct` is 16 bytes with its terminating zero.
- With the check gone, nothing loads. libbpf prints the verifier's log: each instruction with what the verifier knows about every register, such as `R0=map_value_or_null(id=1,map=counts,ks=16,vs=8)` after the lookup. It ends with `R0 invalid mem access 'map_value_or_null'`, then `processed 13 insns (limit 1000000)`. The verifier proved the program could dereference NULL and refused it. A module with the same bug would have been loaded and then crashed.

Exercise — bytes written per program (~30 min): add a second program, `SEC("tracepoint/syscalls/sys_exit_write")`, that adds every successful `write()`'s return value to a second map, `written`. Its argument's layout is in `/sys/kernel/tracing/events/syscalls/sys_exit_write/format`: copy it into a struct, where `ret` sits at offset 16. In the loader, attach every program with `bpf_object__for_each_program()` and print a second column. Look `written` up with the key the map walk gave you, not a C++ string: a key is 16 raw bytes, and the kernel zero-filled the bytes after the name. With `dd if=/dev/zero of=/dev/null bs=1 count=200000` running, `dd`'s row should read `400048` calls and `200101` bytes: the 200,000 bytes, plus about 100 for dd's closing report on stderr. Answer: `day22/solution/`. Then read Ubuntu's `/usr/sbin/runqlat.bt` and find the case it handles that `runqlat.bt` here doesn't: a task preempted while running goes straight back to waiting.

Windows: ETW is the built-in tracing counterpart. Microsoft's eBPF for Windows runs the same kind of bytecode after a verifier, PREVAIL, has checked it.

Check: (a) What makes it safe to load your eBPF program into the kernel you're using, when a module needed a VM? (b) bpftrace can also attach to almost any kernel function with `kprobe:` or `fentry:`. Why prefer a tracepoint when one exists?

Kernel rules for a C++ programmer

Kernel C looks familiar, but the ground rules underneath are different. Reread this page whenever something "obviously fine" crashes.

In user-space C++	In the kernel
C++14	C11 with GNU extensions (`-std=gnu11`): no classes, templates, overloading or references; macros and `typeof` do that work
`new`/`delete`, `std::vector`	`kmalloc`/`kfree`, `kzalloc`; `kmalloc_obj(*p)` sizes the allocation from the pointer's type like `new` does (7.x, `GFP_KERNEL` unless you pass flags); `kvmalloc` for big buffers; `devm_kzalloc` for per-device data; a `kmem_cache` for many objects of one size
`printf`, `std::cout`	`pr_info`, `dev_info`, `dev_dbg` into the kernel log. `%p` prints a hashed address, `%px` the real one (it leaks the kernel's layout), `%pI4` an IPv4 address, `%pe` an error pointer by name
exceptions	a negative errno (`-ENOMEM`); `ERR_PTR`, `IS_ERR` and `PTR_ERR` pass one through a pointer
RAII, `std::lock_guard`, `std::unique_ptr`	`goto` unwinding; `devm_`/`pcim_` resources, freed when the driver unbinds; or `guard(mutex)(&lock)` and `__free(kfree)` from `linux/cleanup.h`
`std::list`, `std::map`	`struct list_head` + `container_of`; `rbtree`, `xarray`, `hashtable.h`
virtual functions	structs of function pointers: `file_operations`, `pci_driver`, `blk_mq_ops`, `net_device_ops`
`std::mutex`, `std::atomic`	`mutex` (may sleep), `spinlock_t` (never sleeps), `atomic_t`, `refcount_t`, RCU for read-mostly data
`std::thread`, `sleep_for`	`kthread_run`, workqueues, threaded interrupt handlers; `msleep` only where sleeping is allowed

In user-space C++	In the kernel
`float`, `double`	none, except between `kernel_fpu_begin()` and `kernel_fpu_end()`
an 8 MB stack, recursion	16 KB per thread on x86-64 (32 KB with KASAN); no recursion, no big local arrays
any pointer you're given	`__user` pointers only through `copy_from_user`, `copy_to_user`, `get_user`, `put_user`
`int`, `long`	`u8` … `u64`; `__le32`/`__be32` with `le32_to_cpu()` for data the hardware defines
`volatile` for hardware	`readl`/`writel` for registers; `READ_ONCE`/`WRITE_ONCE` for shared variables
namespaces	none: prefix every name with your driver's. A module that defines `oops_exit()` won't even compile, because the kernel already declares one
calling any function you can see	a module can call only functions marked `EXPORT_SYMBOL` or `EXPORT_SYMBOL_GPL`; for anything else `modpost` stops the build (Day 9's `pte_offset_map`)
a library's stable API	only the user-space interface (system calls, `/proc`, `/sys`, ioctls) is stable; internal APIs change every release, and a module loads only into the kernel version it was built for
a crash kills your process	an oops kills the thread that hit it, taints the kernel and may leave a lock held forever; in an interrupt handler it is a panic

Seven rules prevent most driver bugs:

1. **Know your context.** In process context you may sleep. In atomic context (interrupt handlers, softirqs, timers, anything under a spinlock or `rcu_read_lock()`) you may not: no mutex, no `GFP_KERNEL`, no `copy_*_user`, no `msleep`. `DEBUG_ATOMIC_SLEEP` reports every violation (Day 11).
2. **Never trust user space.** Check every size, index and pointer it hands you.
3. **Give back everything you take,** on every path, in reverse order: `goto` unwinding, `devm_`, or `cleanup.h`.
4. **Check every return value.** A kernel function that fails returns a negative errno or `NULL`, not an exception.
5. **Assume everything runs at once:** on every CPU, interrupted at any instruction, preempted between any two lines (Day 12).
6. **Develop on a debug kernel.** Lockdep, `DEBUG_ATOMIC_SLEEP`, KASAN and kmemleak turn once-a-month bugs into every-run bugs. Switch a detector off for one file only when you must (`KASAN_SANITIZE_crash.o := n`, Day 6).
7. **Rebuild modules with the kernel.** A module whose version string doesn't match the running kernel is refused; after any kernel change, run `make` in each day's folder again (Day 7).

After Day 22

You can now open any driver in the tree and tell which layer it plugs into and which of these patterns it uses. Directions from here, in the order I'd take them.

Real hardware: your USB device on an ESP32-S3

Day 17 built a vendor-specific USB device in software. An ESP32-S3 makes it real, and Day 17's `usb_echo.ko` drives it unchanged. The S3 has a native USB port that firmware can turn into any device, and Espressif's Arduino core exposes TinyUSB's vendor class as `USBVendor`. Only the S2 and S3 can do this: the original ESP32 has no USB controller, and the C3 and C6 have only a fixed serial/JTAG port.

Firmware, in the Arduino IDE. On Ubuntu 24.04 the IDE's AppImage needs `sudo apt install libfuse2t64`, and the serial port needs `sudo usermod -aG dialout $USER` (log out and back in). Install Espressif's "esp32" boards in the Boards Manager (tested with 3.3.12) and open `~/os/lab/esp32/echo/echo.ino`. It's about 40 lines: it sets the IDs to `0xcafe`/`0x0001`, and whenever a packet arrives on its bulk OUT endpoint, it writes the same bytes to its bulk IN endpoint and calls `flush()`; without the flush, TinyUSB holds them until 64 bytes pile up. In the Tools menu choose Board "ESP32S3 Dev Module", **USB Mode "USB-OTG (TinyUSB)"** and USB CDC On Boot "Disabled". The default USB Mode, "Hardware CDC and JTAG", doesn't let a sketch define a device, and CDC on boot would add a serial interface beside yours; the sketch refuses to compile with either. Upload as usual. From then on, the native USB port is your device, not a serial port: if your board has a second port marked UART, upload and read the log (`echoed 13 bytes`, 115200 baud) through that one. With only the native port, hold BOOT while you press RESET before each upload.

See it from Linux:

```
sudo apt install -y libusb-1.0-0-dev
lsusb -d cafe:0001                    # your board, with your IDs
echo 'SUBSYSTEM=="usb", ATTR{idVendor}=="cafe", MODE="0666"' | sudo tee /etc/udev/rules.d/50-cafe.rules
sudo udevadm control --reload && sudo udevadm trigger   # then replug the board
cd ~/os/lab/esp32 && g++ -std=c++14 -Wall -o usb_test usb_test.cpp -lusb-1.0 && ./usb_test
```

`usb_test` finds the bulk endpoints the way `usb_find_common_endpoints()` does in the kernel, claims interface 0, sends "hello device" and reads the echo. Against Day 17's gadget it printed `interface class 0xff, bulk OUT 0x02, bulk IN 0x81` and `sent 13 bytes, got 13 back: hello device`; the ESP32's endpoint numbers may differ, which is why the program looks them up.

Then your driver, in the VM:

```
QEMU_EXTRA="-device qemu-xhci -device usb-host,vendorid=0xcafe,productid=0x0001"
~/os/lab/setup/run.sh
insmod /lab/day17/usb_echo.ko          # sent 13 bytes, device echoed 13: hello device
```

Pull the cable and the VM prints `unplugged`. Your Day 17 solution's `send` file works the same way, under the board's interface in `/sys/bus/usb/devices`.

Grow it: a `/dev` node with `usb_register_dev()`; asynchronous URBs instead of `usb_bulk_msg()`; a vendor control request that switches the board's LED (`Vendor.onRequest()` on the ESP32, `usb_control_msg()` in the driver); and unplugging while a program has the device open.

`drivers/usb/usb-skeleton.c` shows each of these. To see every packet, `cat /sys/kernel/debug/usb/usbmon/0u` in the VM.

Rust: the same drivers, checked by the compiler

7.3 ships Rust samples for much of what you wrote in C, and real drivers in Rust: `nova-core` for NVIDIA GPUs, `tyr` for Arm Mali GPUs, Android's Binder, a cpufreq driver. Rust can't join the lab kernel: Kconfig allows Rust with KASAN only under clang (`depends on !KASAN || CC_IS_CLANG`), and the lab kernel is built with GCC. So build a second kernel from a second checkout of the same repository, following the kernel's Rust quick start for Ubuntu 24.04:

```
sudo apt install -y rustc-1.85 rust-1.85-src bindgen-0.71 clang lld llvm
cd ~/os && git -C linux worktree add --detach ../linux-rust v7.3
cd linux-rust
R="LLVM=1 RUSTC=rustc-1.85 RUSTDOC=rustdoc-1.85 BINDGEN=bindgen-0.71"
make $R rustavailable                 # Rust is available!
make $R defconfig kvm_guest.config
scripts/config -e RUST -e SAMPLES -e SAMPLES_RUST -e SAMPLE_RUST_MINIMAL \
    -e SAMPLE_RUST_MISC_DEVICE -e SAMPLE_RUST_DRIVER_PCI
make $R olddefconfig
make $R -j$(nproc)                    # 12 min on 4 cores
KERNEL=linux-rust/arch/x86/boot/bzImage QEMU_EXTRA="-device pci-testdev"
~/os/lab/setup/run.sh
dmesg | grep -E "rust|testdev"; ls -l /dev/rust-misc-device
```

The samples are built in, so they run at boot: `rust_minimal: Rust minimal sample (init)`, `rust_misc_device: Initialising Rust Misc Device Sample` with `/dev/rust-misc-device` at `10, 257`, and `rust_driver_pci 0000:00:06.0: pci-testdev data-match count: 1` from a PCI driver bound to QEMU's test device (vendor `1b36`, device `0005`). Unbind and bind it through sysfs as on Day 13. Then read `samples/rust/rust_driver_pci.rs` next to `edu.c`: the same ID table, `probe()`, mapped BAR and register accesses. What changes is what the compiler checks: the BAR mapping can't outlive the device's binding, and a register offset known only at run time must go through `try_write8()`, which returns an error instead of writing out of bounds.

Embedded Linux: when nothing announces the hardware

On a PC, PCI and USB devices announce themselves with IDs. On most ARM boards nothing does: a device tree describes the hardware, and platform drivers match it by its `compatible` string. A good next project is an I2C sensor on a Raspberry Pi: describe it in a device-tree overlay, write the driver with `regmap`, report through IIO, then compare yours with the mainline driver for the same chip (for a BMP280, `drivers/iio/pressure/bmp280-core.c`). Day 21 did the first half of this on an emulated board. Bootlin's free kernel and driver training slides and labs do the rest on a real one.

Contributing

- Follow your subsystem's mailing list on lore.kernel.org; b4 fetches a whole patch series from it, and sends yours.

- KernelNewbies' First Kernel Patch walks through a first cleanup patch. `drivers/staging/` holds drivers that aren't ready yet, many with a `TODO` file of known work.
- The Linux Foundation's LFX Mentorship runs Linux kernel bug-fixing terms with mentors.
- Testing counts too: boot each -rc on your hardware and report what broke.

Optional: xv6, a kernel you can read in a weekend

MIT's xv6 is a teaching Unix of about 6,600 lines for RISC-V. It isn't Linux, but after Day 18 you can read all of it, and MIT's 6.1810 labs have you add system calls, page-table features and copy-on-write fork to it. These commands are from the original Part 1 and weren't re-run for this version:

```
sudo apt install -y gcc-riscv64-linux-gnu binutils-riscv64-linux-gnu qemu-system-misc gdb-multiarch
cd ~/os && git clone https://github.com/mit-pdos/xv6-riscv.git && cd xv6-riscv && make qemu
# inside xv6: ls, then echo hi | wc; quit with Ctrl-a, then x
```

Are you ready? From this course to a kernel job

Not completely. No course can do that. Here is what these 22 days give you, and what's still missing.

What you have, if you did every exercise before opening `solution/`: you've built, booted, traced, broken and fixed a current kernel in every major part: system calls, the scheduler's data, page tables, three allocators, a character device, a PCI driver with interrupts and DMA, a block device, a file system, network and USB drivers, timers and workqueues, containers, a device-tree driver on ARM, and eBPF, using the debugging tools you'd use at work. That's a stronger base than most applicants for junior kernel, driver, BSP and embedded Linux roles bring, and it's what interviews for those roles probe.

What hiring managers also look for:

1. **Upstream patches.** A merged patch is public, reviewed by maintainers and searchable on lore.kernel.org. It's the strongest signal there is, above all when your CV has no kernel job on it yet. Three to five small merged fixes show you can work the way the community works.
2. **Depth in one area.** Teams hire for a subsystem or a product: storage, networking, memory management, graphics, virtualization, security, or the board support for one family of chips. That takes weeks in its code, documentation and mailing list, not one day.
3. **Real hardware.** Embedded and BSP interviews start here: a datasheet and a schematic, a board that doesn't boot, a logic analyzer on an I2C bus. The ESP32-S3 project in After Day 22 is a first step.
4. **Architecture.** Caches and memory ordering, the MMU and TLB, interrupt controllers, arm64 exception levels: more than the days touched. Further reading lists where to learn it.
5. **Fluent C.** Interviews have you write and debug C on a whiteboard. Your C++ carries over, but practice C without RAII: explicit cleanup, macros, pointer arithmetic.

Mid-level and senior roles also ask for years of shipped kernel work. A portfolio shortens that path; it doesn't remove it.

What interviews check, and where you practiced it:

Area	Typical questions	Days
C	`container_of`, linked lists, bit operations, struct layout and alignment, `volatile`	6, 10, 12
Processes and system calls	the path of a system call; process versus thread; `fork`, `exec`, copy-on-write	1, 3, 7, 8
Memory	virtual to physical; page faults; the page cache; `kmalloc` versus `vmalloc`; GFP flags	2, 9, 10, 14
Concurrency	spinlock versus mutex; atomic context; RCU; memory barriers; finding a deadlock	11, 12, 19
Interrupts and deferral	top and bottom halves; threaded handlers; softirqs; workqueues; timers	13, 19
Drivers	the device model; probe and remove; DMA; sysfs; device tree; USB	11, 13, 16, 17, 21
Debugging and performance	reading an oops; KASAN; lockdep; ftrace; eBPF; gdb	6, 7, 18, 22
Containers	namespaces; cgroups; what a container shares with the host	20
Working upstream	patches, mailing lists, maintainers, the release cycle	18

Question bank. Answer each one aloud in about two minutes, then check yourself against the days in brackets. Questions the course doesn't cover come with a short answer.

1. Walk through everything between a program calling `read()` on a file and the bytes landing in its buffer. [1, 4, 7, 14, 15]
2. How does `fork()` of a process using 10 GB stay fast? [2, 9]
3. What's the difference between a process and a thread in Linux? [1, 3, 8]
4. What does the scheduler do when a task wakes up, and how does nice change its share? [8, 22]
5. Translate a virtual address to a physical one by hand. What does the TLB cache? [9]
6. Minor and major page faults: what causes each, and how do you count them for one process? [2, 9]
7. `kmalloc`, `vmalloc` or `alloc_pages`: which, when, and why can't a device DMA into `vmalloc` memory? [10, 13]
8. When must you use `GFP_ATOMIC`? [6, 16, 19]
9. Spinlock or mutex: how do you choose, and what happens if you sleep holding a spinlock? [11, 12, 19]
10. What problem does RCU solve, and what does it cost writers? [12]
11. Why isn't `volatile` enough for shared variables or device registers? [12, 13]
12. A deadlock happens once a month in production. How do you find it? [12]
13. An interrupt needs 5 ms of work. Where does each part of that work run? [13, 19]
14. Why can't an interrupt handler call `copy_to_user()`? [11, 19]
15. You plug in a USB device. What happens before your driver's `probe()` runs? [17, 21]
16. How does a driver find its device on a board without PCI? [21]

17. Coherent or streaming DMA: what's the difference? [13] What does an IOMMU add? *An IOMMU translates the addresses a device uses through page tables of its own, so the device reaches only memory the kernel mapped for it. Scattered pages can look contiguous to the device, a buggy or hostile device can't write anywhere else, and a VM can be given a device safely (VFIO).*
18. Why `readl()` and `writel()` instead of a pointer? [13]
19. What does `devm_` save you, and when is it the wrong tool? [13, 21] *`devm_` resources are freed when the driver unbinds, but objects user space still holds, such as an open file on your device, can outlive the binding. Memory those use must be reference-counted, not `devm_`-allocated.*
20. You get an oops: `RIP: mydrv_read+0x42/0x90`. What do you do next? [6, 18]
21. A service got slower after a kernel upgrade. How do you find where the time went? [7, 22] *Also: `perf top` and `perf record -g` for CPU time, off-CPU analysis for time spent waiting, and `git bisect` between the two kernel versions.*
22. How do containers isolate processes without a hypervisor, and where does that isolation end? [20]
23. What stops a buggy eBPF program from crashing the kernel? [22]
24. What happens when the machine runs out of memory? *Reclaim comes first: the kernel drops clean page cache, writes back dirty pages and swaps out anonymous memory. When that can't free enough, the OOM killer kills the process with the highest `oom_score`, mostly its memory use adjusted by `oom_score_adj`. A cgroup that hits `memory.max` gets an OOM kill inside the group instead.*
25. What is a zombie process, and how do you get rid of one? *A process that has exited, but whose parent hasn't collected its exit status with `wait()`. It holds no memory, only its `task_struct` and PID. You can't kill it: make the parent call `wait()`, or end the parent so that init adopts and reaps the zombie.*
26. How does a patch get into Linux, and what is the merge window? [18]
27. A reviewer asks you to split your patch into three. Why? *One logical change per patch: each is easier to review, can be reverted on its own, and lets `git bisect` pin a regression on it. Every patch in the series must build and work by itself.*

At the whiteboard, in C:

- Write `container_of()`, and say why it needs `offsetof()`.
- Reverse a singly linked list; detect a cycle in one.
- Round `x` up to a multiple of a power of two, `(x + a - 1) & ~(a - 1)`; test for a power of two, `x && !(x & (x - 1))`; count the set bits.
- Write a ring buffer for one producer and one consumer without a lock, and place its barriers (`Documentation/core-api/circular-buffers.rst` has the kernel's answer).
- Find the bug in a function that calls `kmalloc(GFP_KERNEL)` under a spinlock, returns early without unlocking, ignores what `copy_from_user()` returned, or doesn't cancel its work item in `remove()`. These are the seven rules on the Kernel rules page.

A portfolio. Interviewers read code and git history. Make each project public, with a README that shows it running:

1. **Your ESP32-S3 device, finished** (1–2 weeks): the "Grow it" list in After Day 22, ending with an unplug while a program has the device open.

2. **A sensor on a real board** (2 weeks): an I2C or SPI sensor on a Raspberry Pi, described in a device-tree overlay, with `regmap` and IIO, then compared line by line with the mainline driver for the same chip.
3. **An eBPF tool that answers a real question** (1 week), such as block I/O latency per cgroup, with libbpf and CO-RE, starting from libbpf-bootstrap.
4. **Upstream patches** (ongoing), the item that counts most. Start with Contributing in After Day 22, then real bugs: syzbot's dashboard lists crashes with reproducers, and a fix that carries `Reported-by: syzbot` makes a strong interview story. Patches that only satisfy checkpatch are welcome in `drivers/staging` and nowhere else.

Write a short post about each one. It becomes your answer to "tell me about something you built".

Twelve weeks after the course, at about 10 hours a week:

- **Weeks 1–2:** finish the ESP32-S3 driver and publish it.
- **Weeks 3–4:** choose the area you want to be hired for, from job ads you'd actually apply to. Read its `Documentation/`, follow its list on lore.kernel.org, and build and run its kselftests.
- **Weeks 5–8:** patches. Staging first, then a documentation fix, then a real bug from syzbot or from a warning you hit yourself. Learn `git send-email` or `b4 send`, and how to answer review: reply inline, and send a v2 with a changelog below the `---` line.
- **Weeks 9–10:** the second project, in your chosen area.
- **Weeks 11–12:** interviews. The question bank aloud, C on paper, two or three recent LWN articles from your area ready to discuss, then apply. Watch for LFX Linux Kernel Mentorship terms; a completed term is a strong line on a CV.

Where the jobs are. Chip makers write drivers for their CPUs, GPUs, SoCs and network and storage controllers. Device makers (phones, cars, routers, industrial and medical equipment) need board support. Cloud providers have kernel, virtualization and networking teams. Linux distributions maintain and backport kernels, and embedded consultancies do all of the above for clients. Search for Linux kernel engineer, device driver engineer, BSP engineer, embedded Linux engineer and systems software engineer. Many postings list upstream contributions as a plus.

Further reading

The sources worth your time after this course, by what you'll use them for.

Keep open while you work

- Elixir: every identifier in the tree, cross-referenced. It's the fastest way to see how other drivers call an API.
- docs.kernel.org, and lore.kernel.org for every mailing list's archive. Search a function's name on lore to find the patch and the discussion that introduced it.
- LWN.net: the weekly kernel page, and the kernel index, which sorts two decades of articles by topic. Articles open to everyone a week after they appear; a subscription pays for the rest.
- KernelNewbies LinuxChanges: what each release changed, in plain words.

Books for current kernels

- Kaiwan N. Billimoria, *Linux Kernel Programming*, 2nd edition (Packt, 2024; Linux 6.1 LTS): the book closest to this course, deeper on modules, memory, scheduling and synchronization.
- John Madieu, *Linux Device Driver Development*, 2nd edition (Packt, 2022): embedded drivers, after Day 21: device tree, GPIO, I2C and SPI, regmap, IIO, DMA, input.
- Kaiwan N. Billimoria, *Linux Kernel Debugging* (Packt, 2022): KASAN, UBSAN, KCSAN, ftrace, kprobes, oops analysis, kdump and crash.
- Frank Vasquez and Chris Simmonds, *Mastering Embedded Linux Development*, 4th edition (Packt, 2025; Linux 6.6, Yocto 5.0): the rest of an embedded product: toolchains, bootloaders, Buildroot, Yocto, updates.
- Brendan Gregg, *Systems Performance*, 2nd edition (2020), and *BPF Performance Tools* (2019): methods and tools for finding where time goes. Ubuntu's `/usr/sbin/*.bt` scripts come from the second.
- Liz Rice, *Learning eBPF* (O'Reilly, 2023): eBPF programming with libbpf and CO-RE. Isovalent gives the PDF away.
- Michael Kerrisk, *The Linux Programming Interface* (2010): everything user space can ask of the kernel, still the reference for Days 1–4 and 20.
- Lorenzo Stoakes, *The Linux Memory Manager* (No Starch Press): about 1,300 pages on memory management, by one of its maintainers, written against Linux 6.0. It's in early access now; the print edition is announced for spring 2027.

Classics: the ideas hold, the APIs don't

- Robert Love, *Linux Kernel Development*, 3rd edition (2010): the shortest clear tour of the core.
- Daniel P. Bovet and Marco Cesati, *Understanding the Linux Kernel*, 3rd edition (2005): x86 internals in depth, from the 2.6 era.
- Jonathan Corbet, Alessandro Rubini and Greg Kroah-Hartman, *Linux Device Drivers*, 3rd edition (2005), free from LWN. Check each function in Elixir before you copy it.
- Mel Gorman, *Understanding the Linux Virtual Memory Manager* (2004), free: still the clearest account of why memory management works the way it does.

Free and deep

- Paul E. McKenney, *Is Parallel Programming Hard, And, If So, What Can You Do About It?*: memory ordering, locking and RCU, from RCU's author.
- Ulrich Drepper, *What Every Programmer Should Know About Memory* (2007): caches, TLBs and NUMA, and why access patterns matter.
- Arm's free Learn the architecture guides for A-profile: start with "AArch64 Exception Model" and "AArch64 memory management", the arm64 side of Days 9, 13 and 21. For x86, volume 3 of Intel's Software Developer's Manuals, the System Programming Guide, is the reference for paging, interrupts and the APIC.
- Bootlin's training materials: slides and labs for the kernel, drivers, Yocto and Buildroot, on real boards.
- linux-insides: boot, interrupts and system calls on x86-64, followed through the source.
- libbpf-bootstrap and eBPF Docs: starting points for real eBPF tools, and a reference for every program type, map and helper.
- Talks from Linux Plumbers Conference, Kernel Recipes and the Embedded Linux Conference, recorded online: maintainers explaining their own subsystems.

Courses and mentorship

- The Linux Foundation's free A Beginner's Guide to Linux Kernel Development (LFD103), by Shuah Khan: from setting up to a first patch.
- The LFX Linux Kernel Mentorship Program: remote, mentored terms, including a bug-fixing track.
- syzbot: kernel crashes found by fuzzing, most with a reproducer, waiting for someone to fix them.

Answer key

Day 1. (a) A trap. The `syscall` instruction switches the CPU to kernel mode and jumps to the entry point the kernel registered at boot, `entry_SYSCALL_64`, which you saw at the bottom of the sleeping process's stack. The kernel saves the registers, looks the number up in its system call table (Day 7 adds an entry), runs the handler, and returns to user mode with the result in a register. That round trip, plus security mitigations, is the ~100 ns. (b) Between `fork` and `exec` the child still runs your program, so it can set itself up first: the shell opens `out.txt` and moves it onto fd 1, then execs `ls`, which never knows. Your Day 4 exercise does exactly that.

Day 2. (a) A big block gets its own mapping so that `free` can hand it straight back to the kernel instead of leaving a hole in the heap. glibc's threshold starts at 128 KB and adjusts as the program runs. (b) Page faults. `mmap` only set up the mapping; the first touch of each page faulted, and the kernel filled the page from the page cache, reading the disk if the data wasn't there yet. The second `cat` found every page already cached: Linux keeps file data in spare RAM (`buff/cache`) and gives it back when programs need memory, so watch the `available` column, not `free`.

Day 3. (a) `++counter` is load, add, store, so the two threads overwrite each other's updates. An uncontended lock is one atomic compare-and-swap in user space; only when it fails does the thread call `futex` to sleep, and unlock calls `futex` to wake someone only if a thread is waiting. (b) A handler can interrupt the program anywhere, even inside `printf` or `malloc` while they hold a lock, so it may only call async-signal-safe functions; setting a `volatile sig_atomic_t` flag is the safe pattern. SIGKILL (and SIGSTOP) can't be caught, blocked or ignored, so there is always a way to stop any process.

Day 4. (a) Not necessarily. `write` returns once the data is in the page cache, and the kernel writes dirty pages to disk later, after about 30 seconds by default. Only `fsync` (or `fdatasync`, or opening with `O_SYNC`) guarantees it, which is why it's so much slower. (b) A process still holds a deleted file open, often a log, so its blocks stay allocated with no name `du` can see. Find it with `sudo find /proc/*/fd -lname '*(deleted)'` (or `lsof +L1`), then restart that process or truncate the file through `/proc/<pid>/fd/<n>`.

Day 5. (a) `.config` holds every Kconfig choice: each feature or driver is built in (`y`), built as a module (`m`) or left out, plus numeric settings. It decides which files compile and which `#ifdef CONFIG_...` blocks exist. Building every driver in would make a huge kernel that takes hours to compile, keeps code for hardware you don't have in RAM, and runs every driver's init at boot (defconfig already ran 728). Distributions build most drivers as modules and load only what the hardware needs. (b) The kernel panics with `Attempted to kill init!`: PID 1 must always exist, because orphaned processes are handed to it. Here `panic=-1` reboots at once and QEMU's `-no-`

`reboot` turns that into an exit, so you'd only see the message. `poweroff -f` asks the kernel to power off directly with the `reboot()` system call; plain `poweroff` asks PID 1 to shut down, and our script doesn't listen for that.

Day 6. (a) `crash.c` writes `sensor→value` through a NULL `struct sensor *`. `value` comes after `int id`, at offset 4, so the CPU faults at 0 + 4. Addresses near 0 are never mapped, which is how the kernel recognizes a NULL dereference. The VM survived because the oops happened in process context, inside `insmod`'s system call: the kernel killed that one task, marked itself tainted (D) and carried on. In an interrupt handler, or with `panic_on_oops=1`, the same bug is a panic. (b) Whenever the code can't sleep: interrupt handlers, softirqs and timers, under a spinlock, inside `rcu_read_lock()`, or with preemption or interrupts off. `GFP_KERNEL` may sleep to reclaim memory; `GFP_ATOMIC` never sleeps, may use emergency reserves, and fails sooner, so handle `NULL`. Day 16's `xmit` is an example.

Day 7. (a) `do_syscall_64` is marked `noinstr`. It runs right after the CPU enters the kernel, before RCU and context tracking know the kernel is running, so a tracer callback there could bring the machine down. `noinstr` code sits in its own section without the `__fentry__` call that ftrace patches, so it isn't in `available_filter_functions`. `__do_sys_getppid` is ordinary kernel code. (b) A system call is a permanent interface: it needs core maintainers' review and an entry in every architecture's table, and it can never be removed. A module can't add one at all; you rebuilt the kernel. Drivers use what already exists on their own device: `read`, `write`, `ioctl`, `poll` and `mmap` on a device file (Days 10 and 11), sysfs attributes (Days 11 and 13), debugfs for debugging, netlink for networking, and configfs for setup (Day 17).

Day 8. (a) Each nice level is a weight: nice 0 weighs 1024 and nice 19 weighs 15 (`sched_prio_to_weight` in `kernel/sched/core.c`). Runnable tasks on one CPU share it in proportion, so the nice-19 one gets 15 / (1024 + 15), about 1.4%. The trace's slices show the same ratio, 1.7 ms against 119 ms, and the few milliseconds B ran at nice 0, before `renice` reached it, add a little. On an idle CPU nice changes nothing: weights only divide time among runnable tasks, and a lone nice-19 task gets the whole CPU. (b) S is `TASK_INTERRUPTIBLE`: asleep until an event or a signal wakes it. I is `TASK_IDLE`, 0x402 = `TASK_UNINTERRUPTIBLE` (0x2) | `TASK_NOLOAD` (0x400): an uninterruptible sleep that doesn't count toward the load average. Idle kernel workers use it so they don't look like D-state tasks stuck on I/O.

Day 9. (a) A PMD entry can map a whole 2 MB page instead of pointing to a table of 512 PTEs. `pmd_leaf()` is true for such an entry (the PSE bit, 0x80): the physical address is the frame in the entry plus the low 21 bits of the virtual address. The kernel's direct map uses 2 MB and 1 GB pages (`pud_leaf()`), and transparent huge pages can give a process 2 MB pages too. (b) Each process has its own page tables: `mm→pgd`, loaded into CR3 on every context switch, and the walk starts at `pgd_offset(mm, addr)`. The same virtual address goes through different tables to different frames. After `fork()` both processes point at the same frames, read-only, until one of them writes (copy-on-write).

Day 10. (a) The DMA API: `dma_alloc_coherent()` (Day 13) for a buffer the CPU and device share, or `kmalloc()` plus `dma_map_single()` to stream one. Both give a bus address the device can use, inside its DMA mask, with the caches handled. `vmalloc` memory is contiguous only in virtual addresses (Day 10: 256 of 257 pages), and a device sees bus addresses, never kernel virtual ones. (b) `read()` copies every frame from the kernel into the process with `copy_to_user` and costs a system call per

frame: 30 MB a second of extra copying. With `mmap()` the process's page tables point at the driver's buffers, so the program reads frames where the device put them, and only small notifications (`poll`, `ioctl`) cross the boundary. V4L2 video capture works this way.

Day 11. (a) The wait condition was checked without the lock, so when the reader wakes, the data may already be gone: two readers woken by one write, or a `KBUF_IOC_CLEAR` in between. Only a check made while holding the mutex can be trusted, hence the `while` loop. (b) The pointer comes from user space. It may be unmapped, point into kernel memory, or need a page fault to bring the page in. `copy_to_user` checks that the range is user memory, handles a fault (it may sleep), and returns how many bytes it couldn't copy instead of oopsing. On x86 it also briefly lifts SMAP, which otherwise makes the CPU fault on any kernel access to user pages. `memcpy` would crash on a bad pointer, or let a user aim the kernel at kernel memory.

Day 12. (a) `+ 1` is three steps: load, add, store. Both CPUs load the same value, say 41; both add one; both store 42. One increment is lost. `READ_ONCE`/`WRITE_ONCE` only stop the compiler from merging, splitting or caching the accesses; they don't make the three steps one. `atomic_inc()` is a single `lock`-prefixed instruction that holds the cache line for the whole update, and the spinlock lets only one CPU run the sequence at a time. (b) A writer's `synchronize_rcu()` waits until every CPU has passed through a quiescent state, such as a context switch. In a non-preemptible kernel `rcu_read_lock()` only disables preemption, so a reader that sleeps causes a context switch, which RCU counts as "this reader is done", and the writer frees memory the reader still uses. In a preemptible kernel the sleep stalls every grace period instead, and with them all memory freed through RCU. Readers that must sleep use SRCU.

Day 13. (a) Device registers have side effects: writing the factorial register starts a computation, and an interrupt stays raised until you write the ack register. Through a plain pointer the compiler may merge, reorder, split or drop accesses. `readl`/`writel` do exactly one 32-bit access, in program order, and `writel` makes earlier writes to memory (a DMA buffer you filled) visible before the device sees the register write. The `__iomem` type also lets sparse catch a direct dereference. (b) The card works with bus (DMA) addresses, not kernel virtual addresses. A `kmalloc` buffer would need `dma_map_single()` to get one, and it might sit above the card's 28-bit limit (256 MB), forcing a bounce buffer. `vmalloc` memory isn't even physically contiguous. `dma_alloc_coherent()` returns a CPU pointer and a `dma_addr_t` for the same buffer, inside the mask you set.

Day 14. (a) Writes land in the page cache: `echo` copied 6 bytes into a cached page, marked it dirty and returned. Writeback threads (the `kworker` in the trace) send dirty pages to the disk later, after 30 seconds by default (`/proc/sys/vm/dirty_expire_centisecs`), or right away on `sync` and `umount`. (b) It sits between file systems and the driver: it merges adjacent requests, orders them, and balances latency between processes (`mq-deadline`, `bfq`, `kyber`). That paid off on spinning disks, where a seek costs milliseconds. An NVMe SSD has no seek and many deep hardware queues, so scheduling costs more CPU than it saves, and `none` passes requests straight to the driver.

Day 15. (a) The inode is the file itself: type, size, permissions, timestamps and operations, one per file. The dentry is a name in a directory that points at an inode, cached in the dcache; hard links are several dentries for one inode. The `struct file` is one `open()`: it holds the open flags, the `file_operations` and the read position, `f_pos`. Two `open()` calls on one name give two files with separate positions. (b) The shell's current directory holds a reference to the mount and to the

directory's dentry. A mount with references (open files, current or root directories) is busy, and `umount` fails with `EBUSY`. `umount -l` detaches it at once and finishes when the last reference goes away.

Day 16. (a) `ndo_start_xmit` runs in atomic context: from the networking softirq, or with bottom halves disabled, while holding the transmit queue's spinlock. Sleeping is forbidden there, so `GFP_KERNEL` is out. (b) The IP header checksum is the ones'-complement sum of the header's 16-bit words. Addition doesn't depend on order, so swapping the two addresses leaves the sum unchanged. The ICMP checksum does change, because the type went from 8 (request) to 0 (reply), which is why `labnet` recomputes it.

Day 17. (a) `probe()` runs in process context with the device present, configured and locked for it, so it may sleep and do I/O. `disconnect()` also runs in process context, but it is called after the device is gone (cable pulled) or while the driver is being unbound. Any transfer may fail with `-ENODEV`, so it should only stop its own I/O, free what it allocated, and make sure nothing starts new transfers. (b) The gadget would present a standard CDC ACM serial port instead of vendor class 0xff. The kernel's `cdc_acm` driver would claim it by class and create `/dev/ttyACM0` on the host side, with `/dev/ttyGS0` on the gadget side: a standard class needs no driver of yours. To try it, add `CONFIG_USB_CONFIGFS_ACM=y` and `CONFIG_USB_ACM=y` to `lab.config` and rebuild.

Day 18. (a) They cost too much. Generic KASAN checks every memory access and keeps shadow memory (one byte per eight), with what its documentation calls significant performance and memory overheads; here it also doubled each kernel stack to 32 KB. kmemleak keeps metadata for every allocation, scans memory, and reports false positives. Both are debugging tools. Their production cousin is KFENCE, a sampling detector "designed to be enabled in production kernels". (b) Mainline is Linus Torvalds' tree: a two-week merge window, then weekly -rc releases, with a new version every 9 or 10 weeks (7.4-rc2 as this is written). Stable releases (7.3.y) are bug-fix updates to the latest mainline release. Longterm releases, about one a year, get fixes for years (6.18 is the newest). linux-next merges every maintainer's tree daily, as a preview of the next merge window. Base a new patch on the tree of the maintainer who will take it (the `T:` line in `MAINTAINERS`), or on the latest mainline -rc. Never base it on stable or longterm, which only take fixes that are already in mainline.

Day 19. (a) Tasklets run in softirq context, like `timer_list` callbacks, so they can't sleep, and they come with awkward rules: one tasklet never runs on two CPUs at once, it runs on the CPU that scheduled it, and it can't be prioritized against other work. When Linux 6.9 added BH workqueues (`WQ_BH`, `system_bh_wq`), which the workqueue documentation calls "a convenience interface to softirq", to replace them, Linus Torvalds called the tasklet interface "basically deprecated for a long while". New drivers use a threaded interrupt handler (Day 13) or a workqueue when the work may sleep, and a BH workqueue only when it must run in softirq context. (b) `rmmod` frees the module's code and data. A work item still queued or running would make a kworker jump into freed memory: an oops some time later, in a thread that has nothing to do with your module. `cancel_work_sync()` removes a pending item and waits for a running one. The order matters too: the timer queues the work, so it stops first; otherwise it could queue the work again after `cancel_work_sync()` returned. `timer_shutdown_sync()` also refuses any later re-arm, which a callback that re-arms itself needs.

Day 20. (a) Containers share the host's kernel: namespaces change only what their processes see, and cgroups only what they may use. Every system call, file system and network path is attack surface, and a kernel bug reachable from inside a container is a bug in the host. A VM runs its own kernel on virtual hardware, so an attacker must also break the hypervisor, a much smaller interface. That's why public clouds run each customer's containers inside VMs, such as Firecracker microVMs or Kata Containers, or put gVisor's user-space kernel in between. (b) A process's PID can't change, so `unshare(CLONE_NEWPID)` doesn't move the caller: it makes the caller's next child PID 1 of the new namespace. `unshare` must fork and run the command in that child. The child is the namespace's init: orphans inside are reparented to it, and when it exits, the kernel kills every other process in the namespace.

Day 21. (a) A PC's buses describe themselves: PCI devices answer with vendor and device IDs in their configuration space, USB devices send descriptors, and the firmware's ACPI tables describe the rest. An SoC's peripherals sit at fixed addresses on internal buses with nothing to enumerate, and reading random addresses to look for them can hang the bus. So the bootloader hands the kernel a device tree that says what is at which address, with which interrupts, clocks and GPIOs, wired how on this board. Before device trees, each ARM board had a C file of `platform_device` structures under `arch/arm/mach-*`, and a kernel built for one board couldn't boot another. (b) `of_property_read_u32()` reads device-tree properties only. `device_property_read_u32()` goes through the firmware-node layer, so the same call reads a device-tree property, an ACPI `_DSD` property or a software node. Written that way, the driver works unchanged when ACPI describes the device, as on ARM servers.

Day 22. (a) The verifier. Before the program runs, it follows every path through it, tracking what each register holds, and proves that the program ends (bounded loops, at most a million instructions examined), touches only memory it may (its 512-byte stack, its context argument at checked offsets, map values only after a NULL check), and calls only helpers its program type allows, with arguments of the right types. A module is native code with every privilege, so a bug in it corrupts the kernel; a bug in an eBPF program is refused when it loads. The verifier is complex and has had bugs of its own, which is why distributions don't let unprivileged users load eBPF programs (`kernel.unprivileged_bpf_disabled`) and you needed `sudo`. (b) A tracepoint is a hook a maintainer placed on purpose, with named arguments described in its format file, and it stays put as the code around it changes. `kprobe:` and `fentry:` attach to a function by name: an internal function that may be renamed, inlined or given new arguments in the next release, which breaks the tool. Use them where no tracepoint exists. `fentry` is the cheaper of the two: it calls your program directly through a BPF trampoline, with typed arguments from BTF, while a kprobe goes through the kprobe machinery.

Sources and test environment

Where each day's facts come from. Each "Read" line links the exact sections. Behind them:

- OSTEP (*Operating Systems: Three Easy Pieces*), free online, for Days 1–4.
- The kernel's own documentation at docs.kernel.org: the process guides (adding system calls, submitting patches, stable rules, minimal tool versions), the dev-tools pages (KASAN, kmemleak, KFENCE, gdb), the trace guides (ftrace, kprobe events), and the core-api, driver-api, block, filesystems, networking, USB, scheduler and mm pages linked in each day.

- The v7.3 source itself, read for every API the code uses and for the drivers the lab code is modeled on: `drivers/net/dummy.c`, `drivers/block/brd.c`, securityfs in `security/inode.c`, the Loopback gadget function, `samples/kmemleak/`, `samples/rust/`, `include/linux/cleanup.h` and `slab.h`, and `kernel/sched/core.c` for the nice weights.
- QEMU's documentation: the edu device, direct Linux boot, gdb, and the `-virtfs` and `device_add` options.
- LKMPG, kept current with recent kernels, and Linux Kernel Labs, whose lectures explain the concepts well, though their labs target older kernels.
- For the ESP32-S3: Espressif's arduino-esp32 source (the `USBVendor` class, its interface descriptor and the board menus) and TinyUSB's vendor class.

What was run.

- Days 1–4: every command and program, on Ubuntu 24.04.5 user space with a 6.18 kernel. Numbers such as the 130 ns system call and the spinlock timings depend on your machine.
- Days 5–18 and their exercise answers: Linux 7.3 from the `v7.3` tag, configured with `defconfig`, `kvm_guest.config` and `lab.config`, built with GCC 13.3.0, and booted in QEMU 8.2.2 with emulation only (the test machine has no KVM; 4 host cores), with the static BusyBox 1.36.1. The VM had 2 virtual CPUs and 2 GB, and booted in 17 to 19 seconds.
- The Rust kernel in After Day 22: built and booted the same way, with Ubuntu's rustc 1.85 and bindgen 0.71; all three samples ran.
- Days 19 and 20: in the same VM. Day 21: an arm64 kernel from the same tag, configured with `defconfig`, `virt.config` and `mod2noconfig` plus 9p and the function tracer, cross-built with Ubuntu's GCC 13.3 for aarch64 in 13 minutes on the 4 cores, and booted on QEMU's `virt` board (Cortex-A57, 2 CPUs, 1 GB, emulated) in 6 seconds, with Ubuntu's arm64 BusyBox 1.36.1 and dtc 1.7.0. Day 22: bpftrace 0.20.2, clang 18.1.3 and libbpf 1.3.0 from Ubuntu 24.04, as root, on the test machine's own 6.18 kernel.
- The books in Further reading: editions, years and status checked against the publishers' pages in November 2026.
- The ESP32-S3 sketch: compiled with arduino-cli 1.6.0 and Espressif's core 3.3.12, including both of its menu checks. `esp32/usb_test.cpp` ran against Day 17's gadget inside the VM.
- As of 11 November 2026, kernel.org lists 7.3.4 as the latest stable release, 7.4-rc2 as mainline and 6.18 as the newest longterm release; the course stays on 7.3 because in-kernel APIs move.

Addresses, PIDs, IRQ numbers and timings will differ on your machine; their shapes won't.

What wasn't run. KVM itself, since the test machine had none: `run.sh` uses QEMU's standard `-enable-kvm -cpu host` when it can, so the speed-ups this doc mentions are expected, not measured. Nothing ran on a real ESP32-S3: flashing, the board's log, passing a real device into the VM, and the Arduino IDE itself (arduino-cli builds with the same core and compiler). The optional xv6 commands come from the original Part 1 and weren't re-run. Day 22's `kprobe:` and `fentry:` probes: the test machine's kernel offers neither, which is why the day uses tracepoints only; Ubuntu's kernel has both.